黑马程序员™
www.itheima.com

- 黑马程序员

Python 面试宝典

Version 8.1



Copyright©2018 黑马程序员

第一章 内容介绍.....26

第二章 Python 基础.....27

一、 基础语法.....27

1. 输入与输出.....27

2. 条件与循环.....28

3. 文件操作.....30

4. 异常.....32

5. 模块与包.....33

6. Python 特性.....39

7. Linux 基础和 git.....51

二． 数据类型.....55

1. 字典.....55

2. 字符串.....56

3. 列表.....57

4. 元祖.....64

5. 集合.....64

第三章 Python 高级.....65

一． 元类.....65

二． 内存管理与垃圾回收机制.....66

三． 函数.....68

1. 函数参数.....68

2. 内建函数.....70

Copyright©2018 黑马程序员

3.Lambda..... 73
四．设计模式..... 74
1. 单例..... 74
2. 工厂..... 75
3. 装饰器..... 75
4. 生成器..... 76
五．面向对象..... 78
1. 类..... 78
2. 对象..... 78
六．正则表达式..... 80
七． 系统编程..... 82
八． 网络编程..... 92
第四章 前端..... 108
一． Html..... 108
二． Css..... 108
1. 什么是 CSS 初始化？有什么好处？(2018-4-16-lxy)..... 108
2. 简述浮动的特征和清除浮动的方法？(2018-4-16-lxy)..... 108
三． JavaScript..... 109
四． jQurey..... 110
五．vue.js..... 110
第五章 Web..... 110
一． Flask..... 110

Copyright©2018 黑马程序员

二． Django.....121
三． Tornado.....157
第六章 爬虫.....158
一． 常用库与模块.....158
1. 试列出至少三种目前流行的大型数据库的名称:_____、_____、_____,其中您最熟悉的是_____,从_____年开始使用（2018-4-1-ydy）.....158
2. 列举您使用过的 Python 网络爬虫所用到的网络数据包?(2018-4-16-lxy).....158
3. 列举您使用过的 Python 网络爬虫所用到的解析数据包（2018-4-1-ydy）.....158
4. 爬取数据后使用哪个数据库存储数据的，为什么？（2018-4-1-ydy）.....158
5. 你用过的爬虫框架或者模块有哪些？谈谈他们的区别或者优缺点？(2018-4-16-lxy).....161
6. 写爬虫是用多进程好？还是多线程好？ 为什么？(2018-4-16-lxy).....162
7. 常见的反爬虫和应对方法？(2018-4-16-lxy).....162
8. 解析网页的解析器使用最多的是哪几个? (2018-4-16-lxy).....164
9. 需要登录的网页，如何解决同时限制 ip，cookie,session（其中有一些是动态生成的）在不使用动态爬取的情况下？(2018-4-16-lxy).....164
10. 验证码的解决? (2018-4-16-lxy).....164
11. 使用最多的数据库（Mysql，Mongodb，redis 等），对他们的理解？(2018-4-16-lxy)164
12. 字符集和字符编码(2018-4-23-lyf).....165
13. 写一个邮箱地址的正则表达式？(2018-4-23-lyf).....166
14. 编写过哪些爬虫中间件？(2018-4-23-lyf).....166
15. “极验”滑动验证码如何破解？(2018-4-23-lyf).....166

Copyright©2018 黑马程序员

16. 爬的那些内容数据量有多大，多久爬一次，爬下来的数据是怎么存储？（2018-4-20-xhq）..... 167
17. cookie 过期的处理问题？(2018-4-20-xhq)..... 167
18. 动态加载又对及时性要求很高怎么处理？(2018-4-20-xhq)..... 167
19. HTTPS 有什么优点和缺点(2018-4-20-xhq)..... 167
20. HTTPS 是如何实现安全传输数据的。(2018-4-20-xhq)..... 168
21. TTL，MSL，RTT？(2018-4-20-xhq)..... 168
22. 谈一谈你对 Selenium 和 PhantomJS 了解（2018-4-23-xhq）..... 169
23. 代理 IP 里的“透明” “匿名” “高匿”分别是指？(2018-4-23-xhq)..... 170
24. requests 返回的 content 和 text 的区别？..... 170
25. robots 协议(2018-4-23-xhq)..... 171
26. 为什么 requests 请求需要带上 header？(2018-4-23-xhq)..... 171
27. dumps,loads 与 dump,load 的区别？(2018-4-23-xhq)..... 172
28. 通用爬虫 ：通常指搜索引擎的爬虫（2018-4-23-xhq）..... 172
29. requests 使用小技巧(2018-4-23-xhq)..... 173
30. 平常怎么使用代理的 ？(2018-4-23-xhq)..... 173
31. IP 存放在哪里？怎么维护 IP？对于封了多个 ip 的 ,怎么判定 IP 没被封？(2018-4-23-xhq)..... 173
32. 怎么获取加密的数据？（2018-4-23-xhq）..... 174
33. 假如每天爬取量在 5、6 万条数据，一般开几个线程，每个线程 ip 需要加锁限定吗？（2018-4-23-xhq）..... 174
34. 怎么监控爬虫的状态（2018-4-23-xhq）..... 174
二． Scrapy..... 175

Copyright©2018 黑马程序员

1. 描述下 scrapy 框架运行的机制？(2018-4-16-lxy)..... 175
2. 谈谈你对 Scrapy 的理解？(2018-4-23-lyf)..... 175
3. 怎么样让 scrapy 框架发送一个 post 请求（具体写出来）(2018-4-23-lyf)..... 176
4. 怎么判断网站是否更新？(2018-4-23-lyf)..... 176
5. 图片、视频爬取怎么绕过防盗连接，或者说怎么获取正确的链接地址？(2018-4-23-lyf)176
6. 你爬出来的数据量大概有多大？大概多长时间爬一次？(2018-4-23-lyf)..... 177
7. 增量爬取(2018-4-23-lyf)..... 177
8. 爬虫向数据库存数据开始和结束都会发一条消息，是 scrapy 哪个模块实现的？（2018-4-20-xhq）..... 177
9. 爬取下来的数据如何去重，说一下具体的算法依据（2018-4-20-xhq）..... 178
10. Scrapy 的优缺点?（2018-4-23-xhq）..... 179
11. 怎么设置深度爬取?(2018-4-23-xhq)..... 179

三． Scrapy-redis..... 180

1. scrapy 和 scrapy-redis 有什么区别？为什么选择 redis 数据库？(2018-4-16-lxy)..... 180
2. 分布式爬虫主要解决什么问题？(2018-4-16-lxy)..... 180
3. 什么是分布式存储？(2018-4-23-lyf)..... 180
4. 你所知道的分布式爬虫方案有哪些？(2018-4-23-lyf)..... 181
5. 除了 scrapy-redis，有做过其他的分布式爬虫吗？(2018-4-23-lyf)..... 183
6. 在爬取的时候遇到某些内容字段缺失怎么判断及处理？(2018-4-23-lyf)..... 183

四．自定义框架..... 183

第七章 shell 与自动化运维..... 183

第八章 测试..... 183

Copyright©2018 黑马程序员

一、 测试面试题.....183
1. 禅道和 qc 的区别？(2018-4-23-zcz).....183
2. 编写测试计划的目的是(2018-4-23-zcz).....184
3. 测试人员在软件开发过程中的任务是什么(2018-4-23-zcz).....184
4. 您以往的工作中，一条软件缺陷（或者叫 Bug）记录都包含了哪些内容？如何提交高质量的软件缺陷（Bug）记录？(2018-4-23-zcz).....185
5. 简述黑盒测试和白盒测试的优缺点(2018-4-23-zcz).....185
6. 简述常用的 Bug 管理或者用例管理工具,并且描述其中一个工作流程。(2018-4-23-zcz)186
7. 请列出你所知道的软件测试种类，至少 5 项。(2018-4-23-zcz).....186
8. Alpha 测试与 Beta 测试的区别是什么？(2018-4-23-zcz).....186
9. 举例说明什么是 Bug？一个 bug report 应包含什么关键字？(2018-4-23-zcz).....186
第九章 数据库.....187
一． Mysql.....187
1. Python 中操作 Mysql 步骤（2018-3-31-sxd).....187
2. SQL 的 select 语句完整的执行顺序（2018-3-31-sxd）.....188
3. 说一下 Mysql 数据库存储的原理？(2018-4-16-lxy).....191
4. 事务的特性？(2018-4-16-lxy).....191
5. 数据库索引？(2018-4-16-lxy).....192
6. 数据库怎么优化查询效率？(2018-4-16-lxy).....192
7. Mysql 集群的优缺点？(2018-4-16-lxy).....193
8. 你用的 Mysql 是哪个引擎，各引擎之间有什么区别？(2018-4-16-lxy).....193
9. 数据库的优化？(2018-4-16-lxy).....194

Copyright©2018 黑马程序员

10. Mysql 数据库如何分区、分表？(2018-4-16-lxy)..... 195
11. 如何对查询命令进行优化？(2018-4-16-lxy)..... 195
12. Sql 注入是如何产生的，如何防止？(2018-4-16-lxy)..... 196
13. NoSQL 和关系数据库的区别？(2018-4-16-lxy)..... 197
14. Mysql 数据库中怎么实现分页？(2018-4-23-zcz)..... 198
15. sql 语句怎么看效率？(2018-4-23-zcz)..... 198
16. 优化数据库？提高数据库的性能？(2018-4-23-zcz)..... 198
17. 提取数据库中倒数 10 条数据？(2018-4-23-zcz)..... 200
18. 数据库负载均衡(2018-4-23-zcz)..... 200
19. Mysql 数据库的操作?(2018-5-1-lxy)..... 202
20. 数据库的设计？(2018-5-1-lxy)..... 203
21. 存储过程和函数的区别?(2018-5-1-lxy)..... 203
22. Mysql 日志 (2018-5-1-lyf)..... 203
二． MongoDB..... 204
1. 数据库的一些基本操作命令（列举一些常用命令即可）？(2018-4-23-zcz)..... 204
2. Python 中调用 mongo 数据库的包叫什么？(2018-4-23-zcz)..... 205
3. MongoDB (2018-4-23-zcz)..... 205
4. MongoDB 成为优秀的 NoSQL 数据库的原因是什么? (2018-4-23-zcz)..... 207
5. 分析器在 MongoDB 中的作用是什么? (2018-4-23-zcz)..... 208
6. 怎么查看 MongoDB 正在使用的链接? (2018-4-23-zcz)..... 208
7. MySQL 与 MongoDB 本质之间最基本的差别是什么(2018-4-23-zcz)..... 208
8. 使用 MongoDB 的优点(2018-4-23-zcz)..... 209

Copyright©2018 黑马程序员

9. 关于 MongoDB 更多知识请扫描二维码查看(2018-4-23-zcz)....210
三． Redis....210
1. Redis 中 list 底层实现有哪几种？有什么区别？(2018-4-16-lxy)....210
2. 怎样解决数据库高并发的问题？(2018-4-23-zcz)....211
3. Redis 集群实现？(2018-4-23-zcz)....214
4. Redis 数据库，内容是以何种结构存放在 Redis 中的？(2018-4-23-zcz)....214
5. Redis 的并发竞争问题怎么解决？(2018-4-23-zcz)....214
6. MySQL 和 Redis 高可用性体现在哪些方面？(2018-4-23-zcz)....215
7. Redis 和 MongoDB 的优缺点(2018-4-23-zcz)....216
8. Redis 基本类型、相关方法(2018-4-23-zcz)....217
9. Redis 的事务？(2018-4-23-zcz)....218
10. Redis 的使用场景有哪些？(2018-4-23-zcz)....219
11. Redis 默认端口，默认过期时间，Value 最多可以容纳的数据 长度？(2018-4-23-zcz)219
12. Redis 有多少个库？(2018-4-23-zcz)....220
第十章 人工智能....220
一． 数据挖掘....220
1. 1G 的文件，每一行是一个词，词的大小不超过 16 字节，内存限制大小是 1M，返回频数最高的 100 个词。（2018-4-23-xhq）....220
2. 一个大约有一万行的文本文件，每行一个词，要求统计出其中最频繁出现的前 10 个词，请给出思想和时间复杂度分析。（2018-4-23-xhq）....221
3. 怎么在海量数据中找出重复次数最多的一个？（2018-4-23-xhq）....221

Copyright©2018 黑马程序员

4. 给 2 亿个不重复的整数，没排过序的，然后再给一个数，如何快速判断这个数是否在那 2 亿个数当中？ 223
二． 机器学习 223
三．深度学习 223
第十一章 数据结构与算法 223
1. 算法的特征？(2018-4-16-lxy) 223
2. 冒泡排序的思想？(2018-4-16-lxy) 224
3. 快速排序的思想？(2018-4-16-lxy) 224
4. 如何判断单向链表中是否有环？(2018-4-16-lxy) 225
5. 基础的数据结构有哪些？(2018-4-23-lyf) 225
6. 基本的算法有哪些，怎么评价一个算法的好坏？(2018-4-23-lyf) 226
7. 哪种数据结构可以实现递归？(2018-4-23-lyf) 226
8. 你知道哪些排序算法（一般是通过问题考算法）(2018-4-23-lyf) 227
9. 斐波那契数列(2018-4-23-lyf) 227
10. 二叉树如何求两个叶节点的最近公共祖先？(2018-4-23-lyf) 227
11. 两个字符串，如何球公共字符串？(2018-4-23-lyf) 228
12. 找出二叉树中最远结点的距离？(2018-4-23-lyf) 228
13. 写一个二叉树(2018-4-23-lyf) 228
14. 写一个霍夫曼数(2018-4-23-lyf) 229
15. 写一个二分查找(2018-4-23-lyf) 230
16. set 用 in 时间复杂度是多少，为什么？(2018-4-23-lyf) 230
17. 深度优先遍历和广度优先遍历的区别？(2018-4-23-lyf) 231

Copyright©2018 黑马程序员

18. 列表中有 n 个正整数范围在[0 ,1000] ,请编程对列表中的数据进行排序 ;(2018-4-23-lyf) 232
19. 面向对象编程中有组合和继承的方法实现新的类，假设我们手头只有“栈”类，请用“组合”的方式使用“栈”（LIFO）来实现“队列”（FIFO），完成以下代码? (2018-4-23-lyf)232
20. 求和问题，给定一个数组，数组中的元素唯一，数组元素数量 N >2，若数组中的两个数相加和为 m，则认为该数对满足要求，请思考如何返回所有满足要求的数对（要求去重），并给出该算法的计算复杂度和空间复杂度(2018-4-23-lyf)..... 233
21. 写程序把一个单向链表顺序倒过来（尽可能写出更多的实现方法，标出所写方法的空间和时间复杂度）(2018-4-23-lyf)..... 233
22. 有一个长度为 n 的数组 a ,里面的元素都是整数 ,现有一个整数 B ,写程序判断数组 a 中是否有两个元素的和等于 B(尽可能写出更多的实现方法 ,标出所写方法的空间和时间复杂度）(2018-4-23-lyf)..... 234
23. 桶排序（最快最简单的排序）(2018-4-23-lyf)..... 235
24. 青蛙跳台阶问题(2018-4-23-lyf)..... 235
25. 删除排序数组中的重复数字 Remove Duplicates from Sorted Array(2018-4-23-lyf)237
26. 两数之和 Two Sum(2018-4-23-lyf)..... 239
27. 删除元素 Remove Element(2018-4-23-lyf)..... 241
28. 加一 Plus One(2018-4-23-lyf)..... 242
29. 用两个队列如何实现一个栈，用两个栈如何实现一个队列？(2018-4-23-lyf)..... 243
30. 爬楼梯 Climbing Stairs(2018-4-23-lyf)..... 243
31. 落单的数 Single Number(2018-4-23-lyf)..... 244
32. 搜索旋转排序数组 Search in Rotated Sorted Array(2018-4-23-lyf)..... 245

Copyright©2018 黑马程序员

33. 三数之和 3Sum(2018-4-23-lyf)....248
34. 四数之和 4Sum(2018-4-23-lyf)....249
35. 下一个排列 Next Permutation(2018-4-23-lyf)....251
36. 第 K 个排列 Permutation Sequence(2018-4-23-lyf)....252
37. 判断数独是否合法 Valid Sudoku(2018-4-23-lyf)....253
38. 链表求和 Add Two Numbers(2018-4-23-lyf)....255
39. 链表划分 Partition List(2018-4-23-lyf)....257
40. 如何翻转一个单链表？(2018-4-23-lyf)....258
41. 旋转链表(2018-4-23-lyf)....259
42. 两两交换链表中的节点(2018-4-23-lyf)....260
43. 重排链表(2018-4-23-lyf)....261
第十二章 企业真题实战....263
一、 360 面试题....263
1. 请拿出 B 表中的 accd，(A 表中和 B 表中的一样的数据)？(2018-4-16-lxy)....263
2. a = “abbbccc”，用正则匹配为 abccc,不管有多少 b，就出现一次？(2018-4-16-lxy)....263
3. xpath 使用的什么库？(2018-4-16-lxy)....263
4. py2 和 py3 的区别？(2018-4-16-lxy)....264
5. Redis 里面 list 内容的长度？(2018-4-16-lxy)....264
6. 多线程交互，访问数据，如果访问到了就不访问了，怎么避免重读？(2018-4-16-lxy)....264
7. Mysql 怎么限制 IP 访问？(2018-4-16-lxy)....264
8. 带参数的装饰器? (2018-4-16-lxy)....264
二、 妙计旅行面试题....266

Copyright©2018 黑马程序员

1. Python 主要的内置数据类型有哪些？(2018-4-16-lxy)....266
2. print(dir('a'))输出的是什么？(2018-4-16-lxy)....266
3. 给定两个 list，A 和 B，找出相同元素和不同元素？(2018-4-16-lxy)....266
4. 请反转字符串？(2018-4-16-lxy)....266
5. 交换变量 a,b 的值？(2018-4-16-lxy)....267
6. 用 select 语句输出每个城市中心距离市中心大于 20km 酒店数？(2018-4-16-lxy)....267
7. 给定一个有序列表，请输出要插入值 k 所在的索引位置？(2018-4-16-lxy)....267
8. 正则表达式贪婪与非贪婪模式的区别？(2018-4-16-lxy)....267
9. 写出开头匹配字母和下划线，末尾是数字的正则表达式？(2018-4-16-lxy)....267
10. 请说明 HTTP 状态码的用途，请说明常见的状态码机器意义？(2018-4-16-lxy)....267
11. 当输入 http://www.itheima.com 时，返回页面的过程中发生了什么？(2018-4-16-lxy)....268
12. 有一个多层嵌套列表 A=[1,2,[3.4["434",[...]]]]请写一段代码遍历 A 中的每一个元素并打印出来。(2018-5-2-xhq)....268
13. 关系型数据库中，表和表之间有左连接，内连接，外连接，分别解释下他们的含义和区别？(2018-5-2-xhq)....269
14. 如何定时启动你的爬虫项目：（2018-5-2-xhq）....270
15. 什么是 scrapy-redis 中的指纹,是如何去重的？（2018-5-2-xhq）....271
16. 代码优化从哪些方面考虑？有什么想法？（2018-5-2-xhq）....271
17. Django 项目的优化（web 通用）（2018-5-2-xhq）....273
三、 智慧星光面试题....274
1. 定义 A=(“a”, “b”, “c”, “d”),执行 delA[2]后的结果为：D(2018-5-1-lxy)....274

Copyright©2018 黑马程序员

2. String = “{1},{0}”; string = string.format(“Hello”, “Python”),请问将 string 打印出来为（C）(2018-5-1-lxy)....275
3. 定义 A=[1,2,3,4],使用列表生成式[i*i for i in A]生成列表为：B(2018-5-1-lxy)....275
4. 请对 Python 数据结构 Tuple,List,Dict 进行操作(2018-5-1-lxy)....275
5. 请用 Python 内置函数处理以下问题？(2018-5-1-lxy)....276
7. 现在需要从一个简单的登陆网站获取信息 ,请使用 Python 写出简要的登陆函数的具体实现？（登录信息只包含用户名，密码）(2018-5-1-lxy)....278
8. 正则表达式操作(2018-5-1-lxy)....278
四、 壹讯面试题....278
1. Python 中 pass 语句的作用是什么？(2018-5-1-lxy)....278
2. 尽可能写出多的 str 方法？(2018-5-1-lxy)....279
3. 生成一个斐波那契数列？(2018-5-1-lxy)....283
4. 说明一下 os.path 和 sys.path 分别代表什么？(2018-5-1-lxy)....284
5. 什么是 lambda 函数？ 有什么好处？(2018-5-1-lxy)....284
五、 H3C 面试题....284
1. 下列哪个语句在 Python 中是非法的?(B)(2018-5-1-lxy)....284
2. 关于 Python 内存管理,下列说法错误的是(B)(2018-5-1-lxy)....285
3. 下面哪个不是 Python 合法的标识符(b)(2018-5-1-lxy)....285
4. 下列哪种说法是错误的（A）(2018-5-1-lxy)....285
5. 下列表达式的值为 True 的是（C）(2018-5-1-lxy)....285
6. Python 不支持的数据类型有(A)(2018-5-1-lxy)....286
7. 关于 Python 中的复数,下列说法错误的是(C)(2018-5-1-lxy)....286

Copyright©2018 黑马程序员

六、 通联数据 (2018-5-2-lyf)..... 286
1. 说一下你对多线程的看法?..... 286
2. 多线程和多线程有什么区别?..... 286
3. 进程间的数据共享和线程间数据共享?..... 286
4. Redis 数据库结构有那些？..... 288
5. 你一般用 Redis 来做什么？..... 288
6. MonggoDB 中存入了 100 万条数据，如何提高查询速度？..... 288
7. 如何提高并发性能？..... 289
8. 说一下你对 Django 框架的认识？..... 290
9. Flask 框架了解吗，说下 Flask 和 Django 的区别？..... 290
10. 有一个无序数组，如何获取第 K 大的数，说下思路，实现后的时间复杂度？..... 292
11. 归并排序的时间复杂度？..... 292
七、 北京号外科技爬虫面试题（2018-5-2 lyf）..... 294
1. 单引号、双引号、三引号的区别？..... 294
2. 如何在一个 function 里面设置一个全局变量？..... 294
3. 描述 yield 使用场景?..... 294
4. 生成 1~10 之间的整数？..... 295
5. Python 如何生成缩略图？..... 295
6. 列出比较熟悉的爬虫框架，并简要说明？..... 296
7. 列举常见的反爬技术，并给出应对方案？..... 296
8. 网络协议 http 和 https 区别？..... 299
9. 什么是 cookie，session 有什么区别？..... 299

Copyright©2018 黑马程序员

10. Mysql 中 myisam 与 innodb 的区别？ 300

八、 首信 Python 研发面试题（2018-5-2 lyf） 302

1. Python 中 list、tuple、dict、set 有什么区别，主要应用在什么样的场景？并用 for 语句遍历？ 302

2. Python 中静态函数、类函数、成员函数的区别？并写一个示例？ 303

3. 用 Python 语言写一个函数，输入一个字符串，返回倒序结果？ 304

4. 介绍一下 Python 的异常处理机制和自己开发过程中的体会? 304

5. jQuery 库中$()是什么？网上有 5 个 元素，如何使用 jQuery 来选择它们？ 306

6. 写一个 Bash Shell 脚本来得到当前的日期、时间、用户名和当前工作目录? 307

7. Django 中使用 memcached 作为缓存的具体方法？有缺点说明？ 307

九、 微影时代（2018-5-2 lyf） 307

1. 装饰器的理解? 307

2. scrapy 框架，反扒机制，手写中间件? 308

3. HTTP 头有什么字段? 308

4. POST 登录数据方式? 309

5. 加密页面如何解析? 309

6. 爬虫如何定时增量更新数据 310

7. scrapy 怎么设置时间间隔？ 310

8. 爬虫遇到验证码如何解决？ 311

十、 斯沃创智 311

1. 简述 Python 中 is 和= =的区别 （2018-5-2-zcz） 311

2. 简述 read,readline 和 readlines 的区别（2018-5-2-zcz） 312

Copyright©2018 黑马程序员

3. 举例说明创建字典的至少两种方法（2018-5-2-zcz）.....312
4. 简述 Python 里面 search 和 match 的区别（2018-5-2-zcz）.....312
5. Python 代码实现:删除一个 list 里面的重复元素（2018-5-2-zcz）.....313
6. Python 代码中(*args, **kwargs)是什么意思（2018-5-2-zcz）.....314
十一、 天广汇通.....314
1. 说明 os.path 和 sys.path 分别代表什么？（2018-5-2-zcz）.....314
2. 解释一下并行（parallel）和并发（concurrency）的区别（2018-5-2-zcz）.....314
3. 在 Python 中可以实现并发的库有哪些？（2018-5-2-zcz）.....314
4. 如果一个程序需要进行大量的 IO 操作，应当使用并行还是并发？（2018-5-2-zcz）.....315
5. 如果程序需要进行大量的逻辑运算操作，应当使用并行还是并发？（2018-5-2-zcz）.....315
十二、 信德数据.....315
1. 网络七层协议是哪几层？HTTP 协议输入是第几层？（2018-5-2-zcz）.....315
2. 什么是 HTTP 协议？HTTP 请求有哪几种？（2018-5-2-zcz）.....315
3. 什么是 HTTP 代理？作用是什么？（2018-5-2-zcz）.....316
4. 什么是反向代理？作用是什么？（2018-5-2-zcz）.....316
5. HTTPS 和 HTTP 的区别（2018-5-2-zcz）.....316
6. 什么是进程？什么事协程？（2018-5-2-zcz）.....316
7. 什么事死锁？死锁产生的四个必要条件？.....317
8. 什么是内存泄漏？（2018-5-2-zcz）.....317
9. Python 代码中(*args, **kwargs)是什么意思？（2018-5-2-zcz）.....317
十三、 成安.....317
1. Python 中使用%与.format 格式化文本（2018-5-2-zcz）.....317

Copyright©2018 黑马程序员

2. Python 的 logging 模块常用的几个等级？（2018-5-2-zcz）.....317
3. 在 HTTP1.1 中常见的状态码有哪些，如何设置状态码？（2018-5-2-zcz）.....318
4. Python 如何处理上传文件？（2018-5-2-zcz）.....318
5. 用 Python 写一段排序算法。（2018-5-2-zcz）.....319
十四、 博派通达.....320
1. 请列举你使用过的 Python 代码检测工具(2018-5-2-xhq).....320
2. 简述 Python 垃圾回收机制和如何解决循环引用(2018-5-2-xhq).....321
3. 请简述如何编写清晰可读的的代码（2018-5-2-xhq）.....323
4. 请列举出常见的 Mysql 存储引擎（2018-5-2-xhq）.....325
5. InnoDB 有哪些特性（2018-5-2-xhq）.....325
6. 请列出 MySQL 数据库查询的技巧（2018-5-2-xhq）.....325
7. 请简述 SQL 注入的原理及如何在代码层面防止 SQL 注入（2018-5-2-xhq）.....328
8. 请列出常见的 HTTP 头及其作用（2018-5-2-xhq）.....328
9. 请列举常见的 HTTP 状态码及其意义（2018-5-2-xhq）.....330
10. 请简述 RESTfulAPI 设计规范的理解（2018-5-2-xhq）.....331
11. 请简述标准库中 functools.wraps 的作用（2018-5-2-xhq）.....335
十五、 乐飞天下.....335
1. 如何判断一个 python 对象的类型（2018-5-2-xhq）.....335
2. Python 里面如何生成随机数（2018-5-2-xhq）.....335
3. 请写出匹配 ip 的 Python 正则表达式（2018-5-2-xhq）.....335
4. 尽可能多的 str 方法（2018-5-2-xhq）.....336

Copyright©2018 黑马程序员

5. 全局变量和局部变量的区别，如何在 function 里面给一个全局变量赋值（2018-5-2-xhq）.....336
6. Tuple 和 list 的区别，有两个 list b1 = [1,2,3] b2=[2,3,4]写出合并代码（2018-5-2-xhq）.....337
7. 请写出一段代码删除一个 list 里面的重复元素 l =[1,1,2,3,4,5,4](2018-5-2-xhq).....338
8. 写出 list 的交集与差集的代码 b1 =[1,2,3] b2=[2,3,4](2018-5-2-xhq).....338
9. 写一段 Python 代码实现 list 里排 a=[1,2,4,2,45,7,10,5,5,7,8,9,0,3](2018-5-2-xhq).....338
10. D= [i**2 for i in range(1,10)]请写出 D 的输出结果（2018-5-2-xhq）.....339
11. 什么时 lambda 函数，下面这段代码的输出是什么（2018-5-2-xhq）.....339
12. 说说用过的几种爬虫框架及他们的优缺点，用过哪些解析 html 的库（2018-5-2-xhq）339
13. 谈一下对于对线程的理解，对于 cpu 密集性 IO 密集性怎样使用线程，说说线程池，线程锁的用法，有么有用过 multiprocessing 或者 concurrrent.futures?(2018-5-2-xhq).....339
十六、 莉莉丝广告开发工程师初始题目.....341
1. 用递归实现快速排序 quik_sort(A)(2018-5-2-xhq).....341
2. 简述 Python 在异常处理中，else 和 finally 的作用分别是什么？（2018-5-2-xhq）....342
3. 简述 Python GIL 的概念，以及它对 Python 多线程的影响。如何实现一个抓取网页的程序，使用多线程是否比单线程有性能提升，并解释原因（2018-5-2-xhq）.....342
十七、 罗格数据.....343
1. 滑动验证码如何解决（2018-5-2-xhq）.....343
2. ajax 请求页面如何加载（2018-5-2-xhq）.....344
3. Selenium+Phantom JS 的使用(2018-5-2-xhq).....344
4. 数据如何存储，数据库字段如何设计（2018-5-2-xhq）.....345
5. 加密页面如何解析（2018-5-2-xhq）.....346

Copyright©2018 黑马程序员

6. HTTPS 网站如何爬取(2018-5-2-xhq)..... 347
十八、 牧游科技..... 347
1. 函数参数传递，下面程序运行的结果是？（2018-5-2-xhq）..... 347
2. Python 中类方法，静态方法的区别及调用（2018-5-2-xhq）..... 347
3. 类变量，实例变量（2018-5-2-xhq）..... 347
4. 函数式编程与内置函数（2018-5-2-xhq）..... 348
十九、 上海金台灯..... 348
1. 什么是 lambda 函数，它有什么好处（2018-5-2-xhq）..... 348
2. 什么是 Python 的 list and dict comprehensions,写一段代码（2018-5-2-xhq）..... 348
3. Python 里面如何实现 tuple 和 list 的转换（2018-5-2-xhq）..... 349
4. Python 里面如何拷贝一个对象（2018-5-2-xhq）..... 349
5. 写一段 except 的函数（2018-5-2-xhq）..... 349
6. Pyhon 里面 pass 语句的作用是什么？(201-5-2-xhq)..... 349
7. 如何知道 Python 对象的类型？(20185-2-xhq)..... 349
8. Python 中 range()函数的用法(2018-5-2-xhq)..... 350
9. Python re 模块匹配 HTML tag 的的时候，<.*>和<.?>有什么区别(2018-5-2-xhq)..... 350
10. Python 里面如何生成随机数(2018-5-2-xhq)..... 350
11. 如何在 function 里面设置一个全局变量(2018-5-2-xhq)..... 350
12. Python 程序中中文乱码如何解决(2018-5-2-xhq)..... 350
13. Python 的传参是传值还是传址(2018-5-2-xhq)..... 350
14. with 语句的作用,写一段代码(2018-5-2-xhq)..... 351
二十、 钱方好近..... 351

Copyright©2018 黑马程序员

1. 求字符串”是一个 test 字符串”的字符个数字符编码为 utf8(2018-5-2-xhq)..... 351

2. 一个 list 对象 a = [1,2,4,3,2,2,3,4]需要去掉里面重复的的值(2018-5-2-xhq)..... 351

3. 有一个文件 test.txt 里面有数据(2018-5-2-xhq)..... 351

二十一、 西北莜面村..... 352

1. 列举出一些常用的设计模式？(2018-5-11-lxy)..... 352

2. Python 关键字 yield 的用法？(2018-5-11-lxy)..... 354

3. 深拷贝，浅拷贝的区别？(2018-5-11-lxy)..... 354

4. 简化代码？(2018-5-11-lxy)..... 354

5. 解释 list dict tuple set 区别？(2018-5-11-lxy)..... 355

6. 这两个参数什么意思 *args **kwargs，我们为什么要使用他们？(2018-5-11-lxy)..... 355

7. 数据库连表查询？(2018-5-11-lxy)..... 355

二十二、 浙江从泰..... 355

1. 数据库优化？(2018-5-11-lxy)..... 355

2. 反爬虫措施？(2018-5-11-lxy)..... 355

3. 分布式爬虫原理？(2018-5-11-lxy)..... 357

二十三、 tataUFO..... 358

1. 将字符串："k:1|k1:2|k2:3|k3:4"，处理成 Python 字典：{k:1， k1:2， ...}# 字典里的 K 作为字符串处理(2018-5-11-lxy)..... 358

2. 现有字典 d={‘a’:24，‘g’:52，‘l’:12，‘k’:33}请按字典中的 value 值进行排序？(2018-5-11-lxy)..... 358

3. 写一个装饰器？(2018-5-11-lxy)..... 358

4. Python 中可变类型和不可变类型有哪些？(2018-5-11-lxy)..... 359

 Copyright©2018 黑马程序员

二十四、 全品教育.....360
1. Tuple 和 list 区别（2018-5-11-xhq）.....360
2. 这两个参数*args **kwargs 是什么意思（2018-5-11-xhq）.....360
3. Python 里面如何实现 tuple 和 list 的转换（2018-5-11-xhq）.....360
4. Python 里面 range 和 xrange 的区别(2018-5-11-xhq).....360
5. Python 里面 classmethod 和 staticmethod 的区别（2018-5-11-xhq）.....361
6. 如何反向输出序列比如[2,6,5,3],输出为[3,5,6,2](2018-5-11-xhq).....361
7. Python 里面实现删除重复的元素(2018-5-11-xhq).....361
8. Python 里面 copy 和 deepcopy 的区别(2018-5-11-xhq).....361
9. Python 里面的 search 和 match 的区别(2018-5-11-xhq).....362
10. 输出下列代码的结果（2018-5-11-xhq）.....362
11. Python 代码如何得到列表的交集和差集？（2018-5-11-xhq）.....363
12. 请写出一段代码求出 1 到 100 的和？(2018-5-11-xhq).....363
13. Python 中正则表达式提取出字符串中的数字(2018-5-11-xhq).....363
14. 补全下列代码(2018-5-11-xhq).....363
二十五、 名企片爬虫面试题.....364
1. 简述一次完整的 http 的通信过程、常用的响应状态码、http 的无状态性、Cookies 等这些概念(2018-5-11-xhq).....364
2. 说说进程和线程和锁之间的关系(2018-5-17-ydy).....366
3. MySQL 操作：为 person 表的 name 创建普通的索引(2018-5-11-xhq).....367
4. *args and **kwargs 的区别(2018-5-11-xhq).....367
5. 写一个匹配 Email 地址的正则表达式(2018-5-11-xhq).....367

Copyright©2018 黑马程序员

6. 常见的反爬虫措施有哪些？一般怎么克服(2018-5-11-xhq)..... 368
7. 编写爬虫的常用模块或者框架有哪些?请说明一个爬虫的行为步骤(2018-5-11-ydy).....369
8. 排序算法有哪些用 Python 写一种排序算法(2018-5-11-xhq)..... 370
二十六、 欧特咨询..... 372
1. 对 Cookie 的理解，遇到没有 Cookie 登陆的问题(2018-5-11-xhq)..... 372
2. 如何解决验证码的问题，用什么模块，听过哪些人工打码平台(2018-5-11-xhq)..... 372
3. 对于 scrapy_redis 的理解(2018-5-11-xhq)..... 372
4. ip 被封了怎么解决，自己做过 ip 池么？(2018-5-11-xhq)..... 373
二十七、 多来点..... 374
1. 请阐述 Python 的特点(2018-5-8-zcz)..... 374
2. Python 字典参数如何传递？(2018-5-8-zcz)..... 375
3. 请举例说明 Python 闭包的使用场景(2018-5-8-zcz)..... 375
4. 阐述 Python 变量作用域(2018-5-8-zcz)..... 376
二十八、 傲盾..... 376
1. Python 线程和进程的区别？(2018-5-8-zcz)..... 376
2. 如何保证线程安全？(2018-5-8-zcz)..... 376
3. 编程实现 list 转 dict(2018-5-8-zcz)..... 377
4. 编程实现两个 list 的交集、并集、差集(2018-5-8-zcz)..... 377
二十九、 汉迪..... 378
1. 在 Python 中，list，tuple，dict，set 有什么区别，主要应用在什么场景？(2018-5-8-zcz)
..... 378
2. 说明 Session 和 Cookie 的联系(2018-5-8-zcz)..... 378

Copyright©2018 黑马程序员

3. 说明 Session 和 Cookie 的区别(2018-5-8-zcz)..... 379

4. 简述 decorator 的用法(2018-5-8-zcz)..... 380

三十、 大会信统 Python 工程师（2018-5-9-lyf）..... 380

1. 请写出一段 python 代码实现删除一个 list 里面的重复元素..... 380

2. 编程用 sort 进行排序，然后从最后一个元素开始判断..... 381

3. Python 里面如何拷贝一个对象？（赋值，浅拷贝，深拷贝的区别）..... 382

4. Python 里面 match()和 search()的区别？..... 382

5. 用 Python 匹配 HTML tag 的时候，<.*>和<.*?>有什么区别？..... 382

6. Python 里面如何生成随机数？..... 383

三十一、 倍通供应链 信息&数据中心工程师 笔试题（2018-5-9 lyf）..... 383

1. OOP 编程三大特点是什么，多态应用的基础是什么？..... 383

2. 请描述抽象类和接口类的区别和联系？..... 384

3. 请解释委托的定义和作用？..... 384

4. 请描述方法重载与方法重写？..... 385

5. 请用代码实现一个冒泡排序？..... 385

6. 请用代码实现输出：1，2，3，5，8，13，21，34，55，89..... 385

7. 请解释下 TCP/IP 协议和 HTTP 协议？..... 387

8. Python 里面如何实现 tuple 和 list 的转换？..... 387

9. 请写出以下 Linux 的 SHELL 命令？..... 387

三十二、 上海行知道教育 Python 程序员笔试题（2018-5-9 lyf）..... 388

1. Python 如何实现单例模式？请写出两种实现方法？..... 388

2. 什么是 lambda 函数？请举例说明？..... 390

Copyright©2018 黑马程序员

3. 如何反序地迭代一个序列？ 391

4. Python 如何生成随机数？ 392

Copyright©2018 黑马程序员

第一章 内容介绍

该宝典是一份知识点全面又能不断更新，与时俱进的学习手册，不仅收录了作者亲身面试遇到的问题，还收录了近上万名黑马学子面试时遇到的问题。我们会一直不断地更新和充实该宝典，同时也希望读者朋友能够多多提供优质的面试题，也许下一个版本就有你提供的面试题哦。

该面试的最新版发布在黑马论坛：http://bbs.itheima.com/thread-402667-1-1.html

注意：该面试宝典仅供参考，由于作者的知识水平有限加之编写时间仓促因此难免有 bug 的存在，希望大家见谅。

该宝典的一个明确目标是能够让 90%以上的 Python 技术面试题都落到该宝典中，如果您有不错的知识或者面试题，您可以发送到 wangzhenyang@itcast.cn，本人将不胜感激。让天下没有难学的知识，希望你我的努力能帮到更多的莘莘学子。

世间事，很多都可投机取巧，但技术却必须靠日积月累的努力来提高。本宝典更加注重的是知识的掌握，而不仅仅是对面试题的应付。在展示常见的面试问题以及回答技巧的同时还详细讲解了每一道题所包含的知识点，让读者不仅知其然，更知其所以然。

参与该面试宝典收集、整理、编写、审核的人员名单如下：刘鑫洋、刘永峰、辛华侨、张程喆、宋美龙、闫大宇、樊杨俊、郭帅、苏洵、王刚、于健、张晓楠、左文彬、沈夏冬。

Copyright©2018 黑马程序员

第二章 Python 基础

一、基础语法

1. 输入与输出

1.1 代码中要修改不可变数据会出现什么问题？抛出什么异常？(2018-3-29-lxy)

代码不会正常运行，抛出 TypeError 异常。

1.2 a=1,b=2,不用中间变量交换 a 和 b 的值？(2018-3-29-lxy)

方法一：

```
1. a = a+b
2. b = a-b
3. a = a-b
```

方法二：

```
1. a = a^b
2. b =b^a
3. a = a^b
```

方法三：

```
1. a,b = b,a
```

1.3 print 调用 Python 中底层的什么方法?(2018-3-30-lxy)

print 方法默认调用 sys.stdout.write 方法，即往控制台打印字符串。

1.4 下面这段代码的输出结果将是什么？请解释？(2018-3-30-lxy)

```
1.    class Parent(object):
```


http://www.itheima.com Copyright©2018 黑马程序员

```
    x = 1
class Child1(Parent):
    pass
class Child2(Parent):
    pass
print Parent.x, Child1.x, Child2.x
Child1.x = 2
print parent.x, Child1.x, Child2.x
parent.x = 3
print Parent.x, Child1.x, Child2.x
```

结果为：

1 1 1 #继承自父类的类属性 x，所以都一样，指向同一块内存地址。

1 2 1 #更改 Child1，Child1 的 x 指向了新的内存地址。

3 2 3 #更改 Parent，Parent 的 x 指向了新的内存地址。

1.5 简述你对 input()函数的理解?

在 Python3 中，input()获取用户输入，不论用户输入的是什么，获取到的都是字符串类型的。

在 Python2 中有 raw_input()和 input(), raw_input()和 Python3 中的 input()作用是一样的，input()输入的是什么数据类型的，获取到的就是什么数据类型的。

2. 条件与循环

2.1 阅读下面的代码，写出 A0，A1 至 An 的最终值。(2018-3-30-lxy)

```
A0 = dict(zip(('a','b','c','d','e'),(1,2,3,4,5)))
A1 = range(10)
A2 = [i for i in A1 if i in A0]
A3 = [A0[s] for s in A0]
A4 = [i for i in A1 if i in A3]
A5 = {i:i*i for i in A1}
A6 = [[i,i*i] for i in A1]
```

答：

```
A0 = {'a': 1,  'c': 3,  'b': 2,  'e': 5,  'd': 4}
```


Copyright©2018 黑马程序员

```
2.  A1 = [0 ， 1 ， 2 ， 3 ， 4 ， 5 ， 6 ， 7 ， 8 ， 9]
3.  A2 = []
4.  A3 = [1 ， 3 ， 2 ， 5 ， 4]
1.  A4 = [1 ， 2 ， 3 ， 4 ， 5]
2.  A5 = {0: 0 ， 1: 1 ， 2: 4 ， 3: 9 ， 4: 16 ， 5: 25 ， 6: 36 ， 7: 49 ， 8: 64 ， 9: 81}
3.  A6 = [[0 ， 0] ， [1 ， 1] ， [2 ， 4] ， [3 ， 9] ， [4 ， 16] ， [5 ， 25] ， [6 ， 36] , [7 ， 49] , [8 ， 64] [9 , 81]]
```

2.2 range 和 xrange 的区别？(2018-3-30-lxy)

两者用法相同，不同的是 range 返回的结果是一个列表，而 xrange 的结果是一个生成器，前者是直接开辟一块内存空间来保存列表，后者是边循环边使用，只有使用时才会开辟内存空间，所以当列表很长时，使用 xrange 性能要比 range 好。

2.3 考虑以下 Python 代码，如果运行结束，命令行中的运行结果是什么？(2018-3-30-lxy)

```
1.  l = []
2.  for  i  in  xrange(10):
3.      l.append({ ‘num’ :i})
4.  print l
```

在考虑以下代码，运行结束后的结果是什么？

```
1.  l = []
2.  a = { ‘num’ :0}
3.  for i in xrange(10):
4.      a[ ‘num’ ] = i
5.      l.append(a)
6.  print l
```

以上两段代码的运行结果是否相同，如果不相同，原因是什么？

上方代码的结果：

```
1.  [{ ‘num’ :0} , { ‘num’ :1} , { ‘num’ :2} , { ‘num’ :3} , { ‘num’ :4} , { ‘num’ :5} , { ‘num’ :6} , { ‘num’ :7} , { ‘num’ :8} , { ‘num’ :9}]
```

下方代码结果：

```
1.  [{ ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9} , { ‘num’ :9}]
```


Copyright©2018 黑马程序员

原因是：字典是可变对象，在下方的 l.append(a)的操作中是把字典 a 的引用传到列表 l 中，当后续操作修改 a[‘num’]的值的时候，l 中的值也会跟着改变，相当于浅拷贝。

2.4 以下 Python 程序的输出？(2018-3-30-lxy)

```
1. for i in range(5，0，-1):
2.     print(i)
```

答：5 4 3 2 1

3. 文件操作

3.1 4G 内存怎么读取一个 5G 的数据？(2018-3-30-lxy)

方法一：

可以通过生成器，分多次读取，每次读取数量相对少的数据（比如 500MB）进行处理，处理结束后在读取后面的 500MB 的数据。

方法二：

可以通过 linux 命令 split 切割成小文件，然后再对数据进行处理，此方法效率比较高。可以按照行数切割，可以按照文件大小切割。

更多知识请点击网址或扫描二维码。

https://blog.csdn.net/mxgsgtc/article/details/12048919

Copyright©2018 黑马程序员



3.2 现在考虑有一个 jsonline 格式的文件 file.txt 大小约为 10K，之前处理文件的代码如下所示：(2018-3-30-lxy)

```
1. def get_lines():
2.         l = []
3.         with open( ‘file.txt’ , ‘rb’ ) as f:
4.                 for eachline in f:
5.                         l.append(eachline)
6.         return l
7.     if __name__ ==  ‘__main__’ :
8.         for e in get_lines():
9.             process(e) #处理每一行数据
```

现在要处理一个大小为 10G 的文件，但是内存只有 4G，如果在只修改 get_lines 函数而其他代码保持不变的情况下，应该如何实现？需要考虑的问题都有哪些？

```
1.     def get_lines():
2.         l = []
3.         with open( ‘file.txt’ , ‘ rb’ ) as f:
4.                 data = f.readlines(60000)
5.         l.append(data)
6.         yield l
```

要考虑到的问题有：

内存只有 4G 无法一次性读入 10G 的文件，需要分批读入。分批读入数据要记录每次读入数据的位置。分批每次读入数据的大小，太小就会在读取操作上花费过多时间。

Copyright©2018 黑马程序员

3.3 read、readline 和 readlines 的区别? (2018-4-16-lxy)

read:读取整个文件。

readline：读取下一行，使用生成器方法。

readlines：读取整个文件到一个迭代器以供我们遍历。

3.4.补充缺失的代码？(2018-4-16-lxy)

```
1．def print_directory_contents(sPath):
2．    """
3．    这个函数接收文件夹的名称作为输入参数
4．    返回该文件夹中文件的路径
5．    以及其包含文件夹中文件的路径
6．    """
7．    # 补充代码
8．    ------------代码如下--------------------
9．    import os
10．   for sChild in os.listdir(sPath):
11．        sChildPath = os.path.join(sPath, sChild)
12．        if os.path.isdir(sChildPath):
13．            print_directory_contents(sChildPath)
14．        else:
15．            print(sChildPath)
```

4. 异常

4.1 在 except 中 return 后还会不会执行 finally 中的代码？怎么抛出自定义异常？(2018-3-30-lxy)

会继续处理 finally 中的代码；用 raise 方法可以抛出自定义异常。

http://www.itheima.com Copyright©2018 黑马程序员

4.2 介绍一下 except 的作用和用法？(2018-4-16-lxy)

except: #捕获所有异常

except: <异常名>: #捕获指定异常

except:<异常名 1, 异常名 2>： 捕获异常 1 或者异常 2

except:<异常名>,<数据>:捕获指定异常及其附加的数据

except:<异常名 1,异常名 2>:<数据>:捕获异常名 1 或者异常名 2,及附加的数据

5. 模块与包

5.1 常用的 Python 标准库都有哪些？(2018-3-30-lxy)

os 操作系统 ,time 时间 ,random 随机 ,pymysql 连接数据库 ,threading 线程 ,multiprocessing 进程，queue 队列。

第三方库：

django 和 flask 也是第三方库，requests，virtualenv，selenium，scrapy，xadmin，celery，re，hashlib，md5。

常用的科学计算库（如 Numpy，Scipy，Pandas)。

5.2 赋值、浅拷贝和深拷贝的区别？(2018-3-30-lxy)

一、赋值

在 Python 中，对象的赋值就是简单的对象引用，这点和 C++不同，如下所示：

```
16．a = [1,2,"hello",['python', 'C++']]
17．b = a
```

在上述情况下，a 和 b 是一样的，他们指向同一片内存，b 不过是 a 的别名，是引用。

Copyright©2018 黑马程序员

我们可以使用 b is a 去判断，返回 True，表明他们地址相同，内容相同，也可以使用 id()函数来查看两个列表的地址是否相同。

赋值操作(包括对象作为参数、返回值)不会开辟新的内存空间，它只是复制了对象的引用。也就是说除了 b 这个名字之外，没有其他的内存开销。修改了 a，也就影响了 b，同理，修改了 b，也就影响了 a。

二、浅拷贝(shallow copy)

浅拷贝会创建新对象，其内容非原对象本身的引用，而是原对象内第一层对象的引用。

浅拷贝有三种形式:切片操作、工厂函数、copy 模块中的 copy 函数。

比如上述的列表 a；

切片操作：b = a[:] 或者 b = [x for x in a]；

工厂函数：b = list(a)；

copy 函数：b = copy.copy(a)；

浅拷贝产生的列表 b 不再是列表 a 了，使用 is 判断可以发现他们不是同一个对象，使用 id 查看，他们也不指向同一片内存空间。但是当我们使用 id(x) for x in a 和 id(x) for x in b 来查看 a 和 b 中元素的地址时，可以看到二者包含的元素的地址是相同的。

在这种情况下，列表 a 和 b 是不同的对象，修改列表 b 理论上不会影响到列表 a。

但是要注意的是，浅拷贝之所以称之为浅拷贝，是它仅仅只拷贝了一层，在列表 a 中有一个嵌套的 list，如果我们修改了它，情况就不一样了。

比如：a[3].append('java')。查看列表 b，会发现列表 b 也发生了变化，这是因为，我们修改了嵌套的 list，修改外层元素，会修改它的引用，让它们指向别的位置，修改嵌套列表中的元素，列表的地址并未发生变化，指向的都是用一个位置。

三、深拷贝(deep copy)

http://www.itheima.com Copyright©2018 黑马程序员

深拷贝只有一种形式，copy 模块中的 deepcopy()函数。

深拷贝和浅拷贝对应，深拷贝拷贝了对象的所有元素，包括多层嵌套的元素。因此，它的时间和空间开销要高。

同样的对列表 a，如果使用 b = copy.deepcopy(a)，再修改列表 b 将不会影响到列表 a，即使嵌套的列表具有更深的层次，也不会产生任何影响，因为深拷贝拷贝出来的对象根本就是一个全新的对象，不再与原来的对象有任何的关联。

四、拷贝的注意点？

对于非容器类型，如数字、字符，以及其他的“原子”类型，没有拷贝一说，产生的都是原对象的引用。

如果元组变量值包含原子类型对象，即使采用了深拷贝，也只能得到浅拷贝。

5.3__init__ 和__new__的区别？(2018-3-30-lxy)

init 在对象创建后，对对象进行初始化。

new 是在对象创建之前创建一个对象，并将该对象返回给 init。

5.4 Python 里面如何生成随机数？(2018-3-30-lxy)

在 Python 中用于生成随机数的模块是 random，在使用前需要 import. 如下例子可以酌情列举：

random.random()：生成一个 0-1 之间的随机浮点数；

random.uniform(a, b)：生成[a,b]之间的浮点数；

random.randint(a, b)：生成[a,b]之间的整数；

random.randrange(a, b, step)：在指定的集合[a,b)中，以 step 为基数随机取一个数；

Copyright©2018 黑马程序员

random.choice(sequence)：从特定序列中随机取一个元素，这里的序列可以是字符串，列表，元组等。

5.5 输入某年某月某日，判断这一天是这一年的第几天？(可以用 Python 标准库)(2018-3-30-lxy)

```
1. import datetime
2. def dayofyear():
3.     year = input("请输入年份：")
4.     month = input("请输入月份：")
5.     day = input("请输入天：")
6.     date1 = datetime.date(year=int(year)，month=int(month)，day=int(day))
7.     date2 = datetime.date(year=int(year)，month=1，day=1)
8.     return (date1 - date2 + 1).days
```

5.6 打乱一个排好序的 list 对象 alist？(2018-3-30-lxy)

```
1. import random
2. random.shuffle(alist)
```

5.7 说明一下 os.path 和 sys.path 分别代表什么？(2018-3-30-lxy)

os.path 主要是用于对系统路径文件的操作。

sys.path 主要是对 Python 解释器的系统环境参数的操作（动态的改变 Python 解释器搜索路径）。

5.8 Python 中的 os 模块常见方法？(2018-4-16-lxy)

- os.remove()删除文件
- os.rename()重命名文件
- os.walk()生成目录树下的所有文件名
- os.chdir()改变目录

Copyright©2018 黑马程序员

- os.mkdir/makedirs 创建目录/多层目录
- os.rmdir/removedirs 删除目录/多层目录
- os.listdir()列出指定目录的文件
- os.getcwd()取得当前工作目录
- os.chmod()改变目录权限
- os.path.basename()去掉目录路径，返回文件名
- os.path.dirname()去掉文件名，返回目录路径
- os.path.join()将分离的各部分组合成一个路径名
- os.path.split()返回（ dirname(),basename())元组
- os.path.splitext()(返回 filename,extension)元组
- os.path.getatime\ctime\mtime 分别返回最近访问、创建、修改时间
- os.path.getsize()返回文件大小
- os.path.exists()是否存在
- os.path.isabs()是否为绝对路径
- os.path.isdir()是否为目录
- os.path.isfile()是否为文件

5.9 Python 的 sys 模块常用方法？(2018-4-16-lxy)

- sys.argv 命令行参数 List，第一个元素是程序本身路径
- sys.modules.keys() 返回所有已经导入的模块列表
- sys.exc_info() 获取当前正在处理的异常类,exc_type、exc_value、exc_traceback 当前处理的异常详细信息

Copyright©2018 黑马程序员

- sys.exit(n) 退出程序，正常退出时 exit(0)
- sys.hexversion 获取 Python 解释程序的版本值，16 进制格式如：0x020403F0
- sys.version 获取 Python 解释程序的版本信息
- sys.maxint 最大的 Int 值
- sys.maxunicode 最大的 Unicode 值
- sys.modules 返回系统导入的模块字段，key 是模块名，value 是模块
- sys.path 返回模块的搜索路径，初始化时使用 PYTHONPATH 环境变量的值
- sys.platform 返回操作系统平台名称
- sys.stdout 标准输出
- sys.stdin 标准输入
- sys.stderr 错误输出
- sys.exc_clear() 用来清除当前线程所出现的当前的或最近的错误信息
- sys.exec_prefix 返回平台独立的 python 文件安装的位置
- sys.byteorder 本地字节规则的指示器，big-endian 平台的值是'big',little-endian 平台的值是'little'
- sys.copyright 记录 python 版权相关的东西
- sys.api_version 解释器的 C 的 API 版本
- sys.version_info 元组则提供一个更简单的方法来使你的程序具备 Python 版本要求功能

5.10 unittest 是什么？(2018-4-16-lxy)

在 Python 中，unittest 是 Python 中的单元测试框架。它拥有支持共享搭建、自动测试、在测试中暂停代码、将不同测试迭代成一组，等的功能。

Copyright©2018 黑马程序员

5.11 模块和包是什么？(2018-4-16-lxy)

在 Python 中，模块是搭建程序的一种方式。每一个 Python 代码文件都是一个模块，并可以引用其他的模块，比如对象和属性。

一个包含许多 Python 代码的文件夹是一个包。一个包可以包含模块和子文件夹。

6. Python 特性

6.1 Python 是强语言类型还是弱语言类型？(2018-3-30-lxy)

Python 是强类型的动态脚本语言。

强类型：不允许不同类型相加。

动态：不使用显示数据类型声明，且确定一个变量的类型是在第一次给它赋值的时候。

脚本语言：一般也是解释型语言，运行代码只需要一个解释器，不需要编译。

6.2 谈一下什么是解释性语言，什么是编译性语言? (2018-3-30-lxy)

计算机不能直接理解高级语言，只能直接理解机器语言，所以必须要把高级语言翻译成机器语言，计算机才能执行高级语言编写的程序。

解释性语言在运行程序的时候才会进行翻译。

编译型语言写的程序在执行之前，需要一个专门的编译过程，把程序编译成机器语言（可执行文件）。

6.3 Python 中有日志吗?怎么使用? (2018-3-30-lxy)

有日志。

Copyright©2018 黑马程序员

Python 自带 logging 模块 ,调用 logging.basicConfig()方法 ,配置需要的日志等级和相应的参数，Python 解释器会按照配置的参数生成相应的日志。

6.4 Python 是如何进行类型转换的？(2018-3-30-lxy)

内建函数封装了各种转换函数，可以使用目标类型关键字强制类型转换，进制之间的转换可以用 int(‘str’ ，base=’ n’)将特定进制的字符串转换为十进制，再用相应的进制转换函数将十进制转换为目标进制。

可以使用内置函数直接转换的有：

list---->tuple tuple(list)

tuple---->list list(tuple)

6.5 Python2 与 Python3 的区别？(2018-3-30-lxy)

1) 核心类差异

1. Python3 对 Unicode 字符的原生支持。

Python2 中使用 ASCII 码作为默认编码方式导致 string 有两种类型 str 和 unicode，Python3 只支持 unicode 的 string。Python2 和 Python3 字节和字符对应关系为：

python2	python3	表现	转换	作用
str	bytes	字节	encode	存储
unicode	str	字符	decode	显示

2. Python3 采用的是绝对路径的方式进行 import。

Copyright©2018 黑马程序员

Python2 中相对路径的 import 会导致标准库导入变得困难（想象一下，同一目录下有 file.py，如何同时导入这个文件和标准库 file）。Python3 中这一点将被修改，如果还需要导入同一目录的文件必须使用绝对路径，否则只能使用相关导入的方式来进行导入。

3. Python2 中存在老式类和新式类的区别，Python3 统一采用新式类。新式类声明要求继承 object，必须用新式类应用多重继承。

4. Python3 使用更加严格的缩进。Python2 的缩进机制中，1 个 tab 和 8 个 space 是等价的，所以在缩进中可以同时允许 tab 和 space 在代码中共存。这种等价机制会导致部分 IDE 使用存在问题。Python3 中 1 个 tab 只能找另外一个 tab 替代，因此 tab 和 space 共存会导致报错：TabError: inconsistent use of tabs and spaces in indentation.

2) 废弃类差异

1. print 语句被 Python3 废弃，统一使用 print 函数
2. exec 语句被 python3 废弃，统一使用 exec 函数
3. execfile 语句被 Python3 废弃，推荐使用 exec(open("./filename").read())
4. 不相等操作符"<>"被 Python3 废弃，统一使用"!="
5. long 整数类型被 Python3 废弃，统一使用 int
6. xrange 函数被 Python3 废弃，统一使用 range，Python3 中 range 的机制也进行修改并提高了大数据集生成效率
7. Python3 中这些方法再不再返回 list 对象：dictionary 关联的 keys()、values()、items()，zip()，map()，filter()，但是可以通过 list 强行转换：

```
1. mydict={"a":1,"b":2,"c":3}
2. mydict.keys()  #<built-in method keys of dict object at 0x00000000040B4C8>
3. list(mydict.keys()) #['a', 'c', 'b']
```

8. 迭代器 iterator 的 next()函数被 Python3 废弃，统一使用 next(iterator)

Copyright©2018 黑马程序员

9. raw_input 函数被 Python3 废弃，统一使用 input 函数

10. 字典变量的 has_key 函数被 Python 废弃，统一使用 in 关键词

11. file 函数被 Python3 废弃，统一使用 open 来处理文件，可以通过 io.IOBase 检查文件类型

12. apply 函数被 Python3 废弃

13. 异常 StandardError 被 Python3 废弃，统一使用 Exception

3) 修改类差异

1. 浮点数除法操作符“/”和“//”的区别

“ / ” ：

Python2：若为两个整形数进行运算，结果为整形，但若两个数中有一个为浮点数，则结果为浮点数；

Python3:为真除法，运算结果不再根据参加运算的数的类型。

“//” ：

Python2：返回小于除法运算结果的最大整数；从类型上讲，与"/"运算符返回类型逻辑一致。

Python3：和 Python2 运算结果一样。

2. 异常抛出和捕捉机制区别

Python2

```
1. raise IOError, "file error" #抛出异常
2. except NameError, err:   #捕捉异常
```

Python3

```
1. raise IOError("file error") #抛出异常
2. except NameError as err: #捕捉异常
```

3. for 循环中变量值区别

Copyright©2018 黑马程序员

Python2，for 循环会修改外部相同名称变量的值

```
1. i = 1
2. print （'comprehension: ', [i for i in range(5)] ）
3. print （'after: i =', i　） #i=4
```

Python3，for 循环不会修改外部相同名称变量的值

```
1. i = 1
2. print （'comprehension: ', [i for i in range(5)] ）
3. print （'after: i =', i　） #i=1
```

4. round 函数返回值区别

Python2，round 函数返回 float 类型值

```
1. isinstance(round(15.5),int) #True
```

Python3，round 函数返回 int 类型值

```
1. isinstance(round(15.5),float) #True
```

5. 比较操作符区别

Python2 中任意两个对象都可以比较

```
1. 11 < 'test' #True
```

Python3 中只有同一数据类型的对象可以比较

```
1. 11 < 'test' # TypeError: unorderable types: int() < str()
```


Copyright©2018 黑马程序员

4) 第三方工具包差异

我们在 pip 官方下载源 **pypi** 搜索 Python2.7 和 Python3.5 的第三方工具包数可以发现 ,Python2.7 版本对应的第三方工具类目数量是 28523,Python3.5 版本的数量是 12457，这两个版本在第三方工具包支持数量差距相当大。

我们从数据分析的应用角度列举了常见实用的第三方工具包（如下表），并分析这些工具包在 Python2.7 和 Python3.5 的支持情况：

分类	工具名	用途
数据收集	scrapy	网页采集，爬虫
数据收集	scrapy-redis	分布式爬虫
数据收集	selenium	web 测试，仿真浏览器
数据处理	beautifulsoup	网页解释库，提供 lxml 的支持
数据处理	lxml	xml 解释库
数据处理	xlrd	excel 文件读取
数据处理	xlwt	excel 文件写入
数据处理	xlutils	excel 文件简单格式修改
数据处理	pywin32	excel 文件的读取写入及复杂格式定制
数据处理	Python-docx	Word 文件的读取写入
数据分析	numpy	基于矩阵的数学计算库
数据分析	pandas	基于表格的统计分析库

Copyright©2018 黑马程序员

分类	工具名	用途
数据分析	scipy	科学计算库，支持高阶抽象和复杂模型
数据分析	statsmodels	统计建模和计量经济学工具包
数据分析	scikit-learn	机器学习工具库
数据分析	gensim	自然语言处理工具库
数据分析	jieba	中文分词工具库
数据存储	MySQL-python	mysql 的读写接口库
数据存储	mysqlclient	mysql 的读写接口库
数据存储	SQLAlchemy	数据库的 ORM 封装
数据存储	pymssql	sql server 读写接口库
数据存储	redis	redis 的读写接口
数据存储	PyMongo	mongodb 的读写接口
数据呈现	matplotlib	流行的数据可视化库
数据呈现	seaborn	美观的数据可是湖库，基于 matplotlib
工具辅助	jupyter	基于 web 的 python IDE，常用于数据分析
工具辅助	chardet	字符检查工具
工具辅助	ConfigParser	配置文件读写支持
工具辅助	requests	HTTP 库，用于网络访问

Copyright©2018 黑马程序员

5) 工具安装问题

windows 环境

Python2 无法安装 mysqlclient。Python3 无法安装 MySQL-python、 flup、functools32、Gooey、Pywin32、 webencodings。

matplotlib 在 python3 环境中安装报错：The following required packages can not be built:freetype, png。需要手动下载安装源码包安装解决。

scipy 在 Python3 环境中安装报错，numpy.distutils.system_info.NotFoundError，需要自己手工下载对应的安装包，依赖 numpy,pandas 必须严格根据 python 版本、操作系统、64 位与否。运行 matplotlib 后发现基础包 numpy+mkl 安装失败，需要自己下载，国内暂无下载源

centos 环境下

Python2 无法安装 mysql-python 和 mysqlclient 包，报错：EnvironmentError: mysql_config not found，解决方案是安装 mysql-devel 包解决。使用 matplotlib 报错：no module named _tkinter，安装 Tkinter、tk-devel、tc-devel 解决。

pywin32 也无法在 centos 环境下安装。

6.6 关于 Python 程序的运行方面，有什么手段能提升性能？(2018-3-30-lxy)

1、使用多进程，充分利用机器的多核性能

2、对于性能影响较大的部分代码，可以使用 C 或 C++编写

3、对于 IO 阻塞造成的性能影响，可以使用 IO 多路复用来解决

4、尽量使用 Python 的内建函数

5、尽量使用局部变量

Copyright©2018 黑马程序员

6.7 Python 中的作用域？(2018-4-16-lxy)

Python 中，一个变量的作用域总是由在代码中被赋值的地方所决定。当 Python 遇到一个变量的话它会按照这的顺序进行搜索：

本地作用域(Local)--->当前作用域被嵌入的本地作用域(Enclosing locals)--->全局/模块作用域(Global)--->内置作用域(Built-in)。

6.8 什么是 Python？(2018-4-16-lxy)

- Python 是一种编程语言，它有对象、模块、线程、异常处理和自动内存管理，可以加入其他语言的对比。
- Python 是一种解释型语言，Python 在代码运行之前不需要解释。
- Python 是动态类型语言，在声明变量时，不需要说明变量的类型。
- Python 适合面向对象的编程，因为它支持通过组合与继承的方式定义类。
- 在 Python 语言中，函数是第一类对象。
- Python 代码编写快，但是运行速度比编译型语言通常要慢。
- Python 用途广泛，常被用走"胶水语言"，可帮助其他语言和组件改善运行状况。
- 使用 Python，程序员可以专注于算法和数据结构的设计，而不用处理底层的细节。

6.9 什么是 Python 自省？(2018-4-16-lxy)

Python 自省是 Python 具有的一种能力，使程序员面向对象的语言所写的程序在运行时,能够获得对象的类 Python 型。Python 是一种解释型语言，为程序员提供了极大的灵活性和控制力。

更多知识，扫描如下二维码。

http://www.itheima.com Copyright©2018 黑马程序员



6.10 什么是 Python 的命名空间？(2018-4-16-lxy)

在 Python 中，所有的名字都存在于一个空间中，它们在该空间中存在和被操作——这就是命名空间。它就好像一个盒子，每一个变量名字都对应装着一个对象。当查询变量的时候，会从该盒子里面寻找相应的对象。

6.11 你所遵循的代码规范是什么？请举例说明其要求？(2018-4-20-lxy)

PEP8 规范。

1. 变量

常量：大写加下划线 USER_CONSTANT。

私有变量： 小写和一个前导下划线 _private_value。

Python 中不存在私有变量一说，若是遇到需要保护的变量，使用小写和一个前导下划线。但这只是程序员之间的一个约定，用于警告说明这是一个私有变量，外部类不要去访问它。但实际上，外部类还是可以访问到这个变量。

内置变量： 小写，两个前导下划线和两个后置下划线 __class__

Copyright©2018 黑马程序员

两个前导下划线会导致变量在解释期间被更名。这是为了避免内置变量和其他变量产生冲突。用户定义的变量要严格避免这种风格。以免导致混乱。

2. 函数和方法

总体而言应该使用，小写和下划线。但有些比较老的库使用的是混合大小写，即首单词小写，之后每个单词第一个字母大写，其余小写。但现在，小写和下划线已成为规范。

私有方法 ：小写和一个前导下划线

这里和私有变量一样，并不是真正的私有访问权限。同时也应该注意一般函数不要使用两个前导下划线(当遇到两个前导下划线时，Python 的名称改编特性将发挥作用)。

特殊方法 ：小写和两个前导下划线，两个后置下划线

这种风格只应用于特殊函数，比如操作符重载等。

函数参数：小写和下划线，缺省值等号两边无空格

3. 类

类总是使用驼峰格式命名，即所有单词首字母大写其余字母小写。类名应该简明，精确，并足以从中理解类所完成的工作。常见的一个方法是使用表示其类型或者特性的后缀，例如:

SQLEngine，MimeTypes 对于基类而言，可以使用一个 Base 或者 Abstract 前缀 BaseCookie，AbstractGroup

4. 模块和包

除特殊模块 __init__ 之外，模块名称都使用不带下划线的小写字母。

若是它们实现一个协议，那么通常使用 lib 为后缀，例如:

```
import smtplib

import os

import sys
```


http://www.itheima.com Copyright©2018 黑马程序员

5. 关于参数

5.1 不要用断言来实现静态类型检测。断言可以用于检查参数，但不应仅仅是进行静态类型检测。Python 是动态类型语言，静态类型检测违背了其设计思想。断言应该用于避免函数不被毫无意义的调用。

5.2 不要滥用 *args 和 **kwargs。*args 和 **kwargs 参数可能会破坏函数的健壮性。它们使签名变得模糊，而且代码常常开始在不应该的地方构建小的参数解析器。

6. 其他

6.1 使用 has 或 is 前缀命名布尔元素

```
is_connect = True
has_member = False
```

6.2 用复数形式命名序列

```
members = ['user_1', 'user_2']
```

6.3 用显式名称命名字典

```
person_address = {'user_1':'10 road WD', 'user_2' : '20 street huafu'}
```

6.4 避免通用名称

诸如 list, dict, sequence 或者 element 这样的名称应该避免。

6.5 避免现有名称

诸如 os, sys 这种系统已经存在的名称应该避免。

7. 一些数字

一行列数：PEP 8 规定为 79 列。根据自己的情况，比如不要超过满屏时编辑器的显示列数。

一个函数：不要超过 30 行代码，即可显示在一个屏幕类，可以不使用垂直游标即可看到整个函数。

一个类：不要超过 200 行代码，不要有超过 10 个方法。一个模块 不要超过 500 行。

http://www.itheima.com Copyright©2018 黑马程序员

8. 验证脚本

可以安装一个 pep8 脚本用于验证你的代码风格是否符合 PEP8。

7. Linux 基础和 git

7.1 Linux 的基本命令（怎么区分一个文件还是文件夹）(2018-3-30-lxy)

ls -F 在显示名称的时候会在文件夹后添加“/”，在文件后面加“*”。

7.2 日志以什么格式，存放在哪里？(2018-3-30-lxy)

日志以文本可以存储在“/var/log/”目录下后缀名为.log。

7.3 Linux 查看某个服务的端口? (2018-3-30-lxy)

```
1. netstat -anp | grep service_name
```

7.4 ubuntu 系统如何设置开机自启动一个程序? (2018-3-30-lxy)

直接修改/etc/rc0.d ~ /etc/rc6.d 和/etc/rcS.d 文件夹的内容，添加需启动的程序，S 开头的表示启动，K 开头的表示不启动。

7.5 在 linux 中 find 和 grep 的区别(2018-4-16-lxy)

Linux 系统中 grep 命令是一种强大的文本搜索工具，它能使用正则表达式搜索文本，并把匹配的行打印出来。grep 全称是 Global Regular Expression Print，表示全局正则表达式版本，它的使用权限是所有用户。

linux 下的 find：

http://www.itheima.com Copyright©2018 黑马程序员

功能：在目录结构中搜索文件，并执行指定的操作。此命令提供了相当多的查找条件，功能很强大。

语法：find 起始目录寻找条件操作

说明：find 命令从指定的起始目录开始，递归地搜索其各个子目录，查找满足寻找条件的文件并对之采取相关的操作。

简单点说说，grep 是查找匹配条件的行，find 是搜索匹配条件的文件。

7.6 Linux 重定向命令有哪些？有什么区别？(2018-4-16-lxy)

1、重定向>

Linux 允许将命令执行结果重定向到一个文件 ,本应显示在终端上的内容保存到指定文件中。如 :ls > test.txt (test.txt 如果不存在，则创建，存在则覆盖其内容)。

2、重定向>>

>>这个是将输出内容追加到目标文件中。如果文件不存在，就创建文件；如果文件存在,则将新的内容追加到那个文件的末尾，该文件中的原有内容不受影响。

7.7 软连接和硬链接的区别？(2018-4-16-lxy)

软连接类似 Windows 的快捷方式，当删除源文件时，那么软链接也失效了。硬链接可以理解为源文件的一个别名，多个别名所代表的是同一个文件。当 rm 一个文件的时候，那么此文件的硬链接数减 1，当硬链接数为 0 的时候，文件被删除。

7.8 10 个常用的 Linux 命令？(2018-4-20-lxy)

pwd 显示工作路径

ls 查看目录中的文件

Copyright©2018 黑马程序员

cd /home 进入 '/ home' 目录'

cd .. 返回上一级目录

cd ../.. 返回上两级目录

mkdir dir1 创建一个叫做 'dir1' 的目录'

rm -f file1 删除一个叫做 'file1' 的文件'，-f 参数，忽略不存在的文件，从不给出提示。

rmdir dir1 删除一个叫做 'dir1' 的目录'

groupadd group_name 创建一个新用户组

groupdel group_name 删除一个用户组

tar -cvf archive.tar file1 创建一个非压缩的 tarball

tar -cvf archive.tar file1 file2 dir1 创建一个包含了 'file1', 'file2' 以及 'dir1'的档案文件

tar -tf archive.tar 显示一个包中的内容

tar -xvf archive.tar 释放一个包

tar -xvf archive.tar -C /tmp 将压缩包释放到 /tmp 目录下

tar -cvfj archive.tar.bz2 dir1 创建一个 bzip2 格式的压缩包

tar -xvfj archive.tar.bz2 解压一个 bzip2 格式的压缩包

tar -cvfz archive.tar.gz dir1 创建一个 gzip 格式的压缩包

tar -xvfz archive.tar.gz 解压一个 gzip 格式的压缩包

http://www.itheima.com Copyright©2018 黑马程序员



7.9 Linux 关机命令有哪些？(2018-4-20-lxy)

命令	含义
reboot	重新启动操作系统
shutdown –r now	重新启动操作系统，shutdown 会给别的用户提示
shutdown -h now	立刻关机，其中 now 相当于时间为 0 的状态
shutdown -h 20:25	系统在今天的 20:25 会关机
shutdown -h +10	系统再过十分钟后自动关机
init 0	关机
init 6	重启

7.10 git 合并文件有冲突，如何处理? (2018-3-30-lxy)

1、git merge 冲突了，根据提示找到冲突的文件，解决冲突如果文件有冲突，那么会有类似的标记

http://www.itheima.com Copyright©2018 黑马程序员

2、修改完之后，执行 git add 冲突文件名

3、git commit 注意:没有-m 选项 进去类似于 vim 的操作界面，把 conflict 相关的行删除掉

直接 push 就可以了，因为刚刚已经执行过相关 merge 操作了。

二．数据类型

1. 字典

dict:字典，字典是一组键(key)和值(value)的组合，通过键(key)进行查找，没有顺序， 使用大括号”{}”；

应用场景：

dict，使用键和值进行关联的数据;

1.1 现有字典 d={ ‘a’ :24， ‘g’ :52， ‘i’ :12， ‘k’ :33}请按字典中的 value 值进行排序？ (2018-3-30-lxy)

sorted(d.items()，key = lambda x:x[1]) 。

1.2 说一下字典和 json 的区别？(2018-3-30-lxy)

字典是一种数据结构，json 是一种数据的表现形式，字典的 key 值只要是能 hash 的就行，json 的必须是字符串。

1.3 什么是可变、不可变类型？(2018-3-30-lxy)

可变不可变指的是内存中的值是否可以被改变，不可变类型指的是对象所在内存块里面的值不可以改变，有数值、字符串、元组；可变类型则是可以改变，主要有列表、字典。

Copyright©2018 黑马程序员

1.4 存入字典里的数据有没有先后排序？(2018-3-30-lxy)

存入的数据不会自动排序，可以使用 sort 函数对字典进行排序。

1.5 字典推导式？（2018-4-23-lxy)

```
1．d = {key: value for (key, value) in iterable}
```

1.6 现有字典 d={‘a’:24，’g’:52，’l’:12，’k’:33}请按字 典中的 value 值进行排序？ （2018-4-23-lxy）

```
1．sorted(d.items()，key = lambda x:x[1])
```

2. 字符串

str:字符串是 Python 中最常用的数据类型。我们可以使用引号('或")来创建字符串。

2.1 如何理解 Python 中字符串中的\字符？(2018-3-30-lxy)

有三种不同的含义：

1、转义字符 2、路径名中用来连接路径名 3、编写太长代码手动软换行。

2.2 请反转字符串“aStr”?(2018-3-30-lxy)

```
1. print(‘aStr’[::-1])
```

2.3 将字符串"k:1|k1:2|k2:3|k3:4"，处理成 Python 字典：{k:1， k1:2， ...}# 字典里的 K 作为字符串处理(2018-3-30-lxy)

```
1. str1 = "k:1|k1:2|k2:3|k3:4"
2. def str2dict(str1):
3.     dict1 = {}
```


Copyright©2018 黑马程序员

```
    for iterms in str1.split('|'):
        key , value = iterms.split(':')
        dict1[key] = value
    return dict1
```

2.4 请按 alist 中元素的 age 由大到小排序(2018-3-30-lxy)

```
alist [{'name':'a' , 'age':20} , {'name':'b' , 'age':30} , {'name':'c' , 'age':25}]
def sort_by_age(list1):
    return sorted(alist , key=lambda x:x['age'] , reverse=True)
```

3. 列表

list:是 Python 中使用最频繁的数据类型，在其他语言中通常叫做数组，通过索引进行查找，使用方括号” []”，列表是有序的集合。

应用场景：定义列表使用 [ ] 定义，数据之间使用 “ , ” 分割。

列表的索引从 0 开始：索引就是数据在列表中的位置编号，索引又可以被称为下标。

【注意】：从列表中取值时,如果超出索引范围,程序会产生异常。

IndexError: list index out of range

列表的常用操作：

```
1 . name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu"]
```

1) 增加

列表名.insert(index, 数据)：在指定位置插入数据(位置前有空元素会补位)。

```
1 . # 往列表 name_list 下标为 0 的地方插入数据
2 . In [3]: name_list.insert(0, "Sasuke")
3 . In [4]: name_list
4 . Out[4]: ['Sasuke', 'zhangsan', 'lisi', 'wangwu', 'zhaoliu']
5 . # 现有的列表下标是 0-4，如果我们要在下标是 6 的地方插入数据，那个会自动插入到下标为 5 的地方，也就是# 插入到最后
6 . In [5]: name_list.insert(6, "Tom")
7 . In [6]: name_list
8 . Out[6]: ['Sasuke', 'zhangsan', 'lisi', 'wangwu', 'zhaoliu', 'Tom']
```

列表名.append(数据)：在列表的末尾追加数据(最常用的方法)。

```
1 . In [7]: name_list.append("Python")
```


Copyright©2018 黑马程序员

```
2．In [8]: name_list
3．Out[8]: ['Sasuke', 'zhangsan', 'lisi', 'wangwu', 'zhaoliu', 'Tom', 'Python']
```

列表.extend(Iterable)：将可迭代对象中的元素追加到列表。

```
1．# 有两个列表 a 和 b a.extend(b) 会将 b 中的元素追加到列表 a 中
2．In [10]: a = [11, 22, 33]
3．In [11]: b = [44, 55, 66]
4．In [12]: a.extend(b)
5．In [13]: a
6．Out[13]: [11, 22, 33, 44, 55, 66]# 有列表 c 和 字符串 d c.extend(d) 会将字符串 d 中的每个字符拆开作为元素插入到列表
c
7．In [14]: c = ['j', 'a', 'v', 'a']
8．In [15]: d = "python"
9．In [16]: c.extend(d)
10．In [17]: c
11．Out[17]: ['j', 'a', 'v', 'a', 'p', 'y', 't', 'h', 'o', 'n']
```

2）取值和修改

取值：列表名[index] ：根据下标来取值。

```
1．In [19]: name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu"]
2．In [20]: name_list[0]
3．Out[20]: 'zhangsan'
4．In [21]: name_list[3]
5．Out[21]: 'zhaoliu'
```

修改：列表名[index] = 数据：修改指定索引的数据。

```
1．In [22]: name_list[0] = "Sasuke"
2．In [23]: name_list
3．Out[23]: ['Sasuke', 'lisi', 'wangwu', 'zhaoliu']
```

3）删除

del 列表名[index]：删除指定索引的数据。

```
1．In [25]: name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu"]# 删除索引是 1 的数据
2．In [26]: del name_list[1]
3．In [27]: name_list
4．Out[27]: ['zhangsan', 'wangwu', 'zhaoliu']
```

列表名.remove(数据)：删除第一个出现的指定数据。

```
5．In [30]: name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu", "lisi"]# 删除 第一次出现的 lisi 的数据
6．In [31]: name_list.remove("lisi")
7．In [32]: name_list
8．Out[32]: ['zhangsan', 'wangwu', 'zhaoliu', 'lisi']
```


Copyright©2018 黑马程序员

列表名.pop()：删除末尾的数据,返回值: 返回被删除的元素。

```
9 . In [33]: name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu"]# 删除最后一个元素  zhaoliu  并将元素 zhaoliu 返回
10 . In [34]: name_list.pop()
11 . Out[34]: 'zhaoliu'
12 . In [35]: name_list
13 . Out[35]: ['zhangsan', 'lisi', 'wangwu']
```

列表名.pop(index)：删除指定索引的数据，返回被删除的元素。

```
14 . In [36]: name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu"]# 删除索引为 1 的数据 lisi
15 . In [37]: name_list.pop(1)
16 . Out[37]: 'lisi'
17 . In [38]: name_list
18 . Out[38]: ['zhangsan', 'wangwu', 'zhaoliu']
```

列表名.clear()：清空整个列表的元素。

```
19 . In [40]: name_list = ["zhangsan", "lisi", "wangwu", "zhaoliu"]
20 . In [41]: name_list.clear()
21 . In [42]: name_list
22 . Out[42]: []
```

4）排序

列表名.sort()：升序排序 从小到大。

```
23 . In [43]: a = [33, 44, 22, 66, 11]
24 . In [44]: a.sort()
25 . In [45]: a
26 . Out[45]: [11, 22, 33, 44, 66]
```

列表名.sort(reverse=True)：降序排序 从大到小。

```
27 . In [46]: a = [33, 44, 22, 66, 11]
28 . In [47]: a.sort(reverse=True)
29 . In [48]: a
30 . Out[48]: [66, 44, 33, 22, 11]
```

列表名.reverse()：列表逆序、反转。

```
31 . In [50]: a = [11, 22, 33, 44, 55]
32 . In [51]: a.reverse()
33 . In [52]: a
34 . Out[52]: [55, 44, 33, 22, 11]
```

5）统计相关

len(列表名)：得到列表的长度。

Copyright©2018 黑马程序员

```
35．In [53]: a = [11, 22, 33, 44, 55]
36．In [54]: len(a)
37．Out[54]: 5
```

列表名.count(数据)：数据在列表中出现的次数。

```
38．In [56]: a = [11, 22, 11, 33, 11]
39．In [57]: a.count(11)
40．Out[57]: 3
```

列表名.index(数据)：数据在列表中首次出现时的索引，没有查到会报错。

```
41．In [59]: a = [11, 22, 33, 44, 22]
42．In [60]: a.index(22)
43．Out[60]: 1
```

if 数据 in 列表： 判断列表中是否包含某元素。

```
44．a = [11, 22, 33, 44 ,55]
45．if 33 in a:
46．    print("找到了....")
```

6）循环遍历

使用 while 循环：

```
1. a = [11, 22, 33, 44, 55]
2.
3. i = 0
4. while i < len(a):
5.     print(a[i])
6.     i += 1
```

使用 for 循环：

```
1. a = [11, 22, 33, 44, 55]
2. for i in a:
3.     print(i)
```

3.1 下面代码的输出结果将是什么？(2018-3-30-lxy)

```
1. list = ['a' , 'b' , 'c' , 'd' , 'e']
2. print list[10:]
```

下面的代码将输出[] ,不会产生IndexError错误。就像所期望的那样 ,尝试用超出成员的个数的index来获取某个列表的成员。

例如，尝试获取 list[10]和之后的成员，会导致 IndexError。

Copyright©2018 黑马程序员

然而，尝试获取列表的切片，开始的 index 超过了成员个数不会产生 IndexError，而是仅仅返回一个空列表。这成为特别让人恶心的疑难杂症，因为运行的时候没有错误产生，导致 bug 很难被追踪到。

3.2 写一个列表生成式，产生一个公差为 11 的等差数列(2018-3-30-lxy)

```
print([x*11 for x in range(10)])
```

3.3 给定两个列表，怎么找出他们相同的元素和不同的元素? (2018-3-30-lxy)

```
list1 = [1，2，3]
list2 = [3，4，5]
set1 = set(list1)
set2 = set(list2)
print(set1&set2)
print(set1^set2)
```

3.4 请写出一段 Python 代码实现删除一个 list 里面的重复元素?(2018-3-30-lxy)

比较容易记忆的是用内置的 set：

```
l1 = ['b'，'c'，'d'，'b'，'c'，'a'，'a']
l2 = list(set(l1))
print l2
```

如果想要保持他们原来的排序：

用 list 类的 sort 方法：

```
l1 = ['b'，'c'，'d'，'b'，'c'，'a'，'a']
l2 = list(set(l1))
l2.sort(key=l1.index)
print l2
```

也可以这样写：

```
l1 = ['b'，'c'，'d'，'b'，'c'，'a'，'a']
l2 = sorted(set(l1)，key=l1.index)
print l2
```

也可以用遍历：

```
l1 = ['b', 'c', 'd', 'b', 'c', 'a', 'a']
l2 = []
```


Copyright©2018 黑马程序员

```
3.  for i in l1:
4.      if not i in l2:
5.          l2.append(i)
6.  print l2
```

3.5 给定两个 list A ,B，请用找出 A ,B 中相同的元素，A ,B 中不同的元素 (2018-3-30-lxy)

A、B 中相同元素：print(set(A)&set(B))

A、B 中不同元素：print(set(A)^set(B))

3.6 有如下数组 list = range(10)我想取以下几个数组，应该如何切片？ (2018-3-30-lxy)

```
1. [1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9]
2. [1 , 2 , 3 , 4 , 5 , 6]
3. [3 , 4 , 5 , 6]
4. [9]
5. [1 , 3 , 5 , 7 , 9]
```

答：

```
1. [1:]
2. [1:7]
3. [3:7]
4. [-1]
5. [1::2]
```

3.7 下面这段代码的输出结果是什么？请解释？(2018-4-16-lxy)

```
1. def extendlist(val, list=[]):
2.     list.append(val)
3.     return list
4.
5. list1 = extendlist(10)
6. list2 = extendlist(123, [])
7. list3 = extendlist('a')
8.
9. print("list1 = %s" %list1)
```


Copyright©2018 黑马程序员

```
print("list2 = %s" %list2)
print("list3 = %s" %list3)
```

输出结果：

```
list1 = [10, 'a']
list2 = [123]
list3 = [10, 'a']
```

新的默认列表只在函数被定义的那一刻创建一次。当 extendList 被没有指定特定参数 list 调用时，这组 list 的值随后将被使用。这是因为带有默认参数的表达式在函数被定义的时候被计算，不是在调用的时候被计算。

3.8.将以下 3 个函数按照执行效率高低排序(2018-4-16-lxy)

```
def f1(lIn):
    l1 = sorted(lIn)
    l2 = [i for i in l1 if i<0.5]
    return [i*i for i in l2]


def f2(lIn):
    l1 = [i for i in l1 if i<0.5]
    l2 = sorted(l1)
    return [i*i for i in l2]


def f3(lIn):
    l1 = [i*i for i in lIn]
    l2 = sorted(l1)
    return [i for i in l1 if i<(0.5*0.5)]
```

按执行效率从高到低排列 :f2、f1 和 f3。要证明这个答案是正确的，你应该知道如何分析自己代码的性能。Python 中有一个很好的程序分析包，可以满足这个需求。

```
import random
import cProfile
lIn = [random.random() for i in range(100000)]
cProfile.run('f1(lIn)')
cProfile.run('f2(lIn)')
cProfile.run('f3(lIn)')
```


Copyright©2018 黑马程序员

3.9 获取 1~100 被 6 整除的偶数？(2018-4-23-lxy)

```
def A():
    alist = []
    for i in range(1，100):
        if i % 6 == 0:
            alist.append(i)
    last_num = alist[-3:]
    print(last_num)
```

4. 元祖

tuple:元组，元组将多样的对象集合到一起，不能修改，通过索引进行查找，使用括号”()”；

应用场景：把一些数据当做一个整体去使用，不能修改；

5. 集合

set:set 集合，在 Python 中的书写方式的{}，集合与之前列表、元组类似，可以存储多个数据，但是这些数据是不重复的。集合对象还支持 union(联合), intersection(交), difference(差)和 sysmmetric_difference(对称差集)等数学运算.

快速去除列表中的重复元素

```
1．In [4]: a = [11,22,33,33,44,22,55]
2．
3．In [5]: set(a)
4．Out[5]: {11, 22, 33, 44, 55}
```

交集：共有的部分

```
1．In [7]: a = {11,22,33,44,55}
2．In [8]: b = {22,44,55,66,77}
3．In [9]: a&b
4．Out[9]: {22, 44, 55}
```

并集：总共的部分

```
1．In [11]: a = {11,22,33,44,55}
2．In [12]: b = {22,44,55,66,77}
3．In [13]: a | b
```


Copyright©2018 黑马程序员

```
4．Out[13]: {11, 22, 33, 44, 55, 66, 77}
```

差集：另一个集合中没有的部分

```
1．In [15]: a = {11,22,33,44,55}
2．In [16]: b = {22,44,55,66,77}
3．In [17]: b - a
4．Out[17]: {66, 77}
```

对称差集(在 a 或 b 中，但不会同时出现在二者中)

```
1．In [19]: a = {11,22,33,44,55}
2．In [20]: b = {22,44,55,66,77}
3．In [21]: a ^ b
4．Out[21]: {11, 33, 66, 77}
```

第三章 Python 高级

一．元类

1.Python 中类方法、类实例方法、静态方法有何区别？(2018-3-30-lxy)

类方法：是类对象的方法，在定义时需要在上方使用“@classmethod”进行装饰，形参为 cls，表示类对象，类对象和实例对象都可调用；

类实例方法：是类实例化对象的方法，只有实例对象可以调用，形参为 self，指代对象本身；

静态方法：是一个任意函数，在其上方使用“@staticmethod”进行装饰，可以用对象直接调用，静态方法实际上跟该类没有太大关系。

2.Python 中如何动态获取和设置对象的属性？(2018-3-30-lxy)

```
1. if hasattr(Parent，'x'):
2.     print(getattr(Parent，'x'))
3.     setattr(Parent，'x'，3)
4. print(getattr(Parent，'x'))
```


Copyright©2018 黑马程序员

二．内存管理与垃圾回收机制

1. Python 的内存管理机制及调优手段？(2018-3-30-lxy)

内存管理机制：引用计数、垃圾回收、内存池。

引用计数：

引用计数是一种非常高效的内存管理手段， 当一个 Python 对象被引用时其引用计数增加 1， 当其不再被一个变量引用时则计数减 1. 当引用计数等于 0 时对象被删除。

垃圾回收 ：

1. 引用计数

引用计数也是一种垃圾收集机制，而且也是一种最直观，最简单的垃圾收集技术。当 Python 的某个对象的引用计数降为 0 时，说明没有任何引用指向该对象，该对象就成为要被回收的垃圾了。比如某个新建对象，它被分配给某个引用，对象的引用计数变为 1。如果引用被删除，对象的引用计数为 0，那么该对象就可以被垃圾回收。不过如果出现循环引用的话，引用计数机制就不再起有效的作用了

2. 标记清除

如果两个对象的引用计数都为 1，但是仅仅存在他们之间的循环引用，那么这两个对象都是需要被回收的，也就是说，它们的引用计数虽然表现为非 0，但实际上有效的引用计数为 0。所以先将循环引用摘掉，就会得出这两个对象的有效计数。

3. 分代回收

从前面“标记-清除”这样的垃圾收集机制来看，这种垃圾收集机制所带来的额外操作实际上与系统中总的内存块的数量是相关的，当需要回收的内存块越多时，垃圾检测带来的额外操作就越多，而垃圾回收带来的额外操作就越少；反之，当需回收的内存块越少时，垃圾检测就将比垃圾回收带来更少的额外操作。

http://www.itheima.com Copyright©2018 黑马程序员

举个例子：

当某些内存块 M 经过了 3 次垃圾收集的清洗之后还存活时，我们就将内存块 M 划到一个集合 A 中去，而新分配的内存都划分到集合 B 中去。当垃圾收集开始工作时，大多数情况都只对集合 B 进行垃圾回收，而对集合 A 进行垃圾回收要隔相当长一段时间后才进行，这就使得垃圾收集机制需要处理的内存少了，效率自然就提高了。在这个过程中，集合 B 中的某些内存块由于存活时间长而会被转移到集合 A 中，当然，集合 A 中实际上也存在一些垃圾，这些垃圾的回收会因为这种分代的机制而被延迟。

内存池：

1. Python 的内存机制呈现金字塔形状，-1，-2 层主要有操作系统进行操作；

2. 第 0 层是 C 中的 malloc，free 等内存分配和释放函数进行操作；

3. 第 1 层和第 2 层是内存池，有 Python 的接口函数 PyMem_Malloc 函数实现，当对象小于 256K 时有该层直接分配内存；

4. 第 3 层是最上层，也就是我们对 Python 对象的直接操作；

Python 在运行期间会大量地执行 malloc 和 free 的操作，频繁地在用户态和核心态之间进行切换，这将严重影响 Python 的执行效率。为了加速 Python 的执行效率，Python 引入了一个内存池机制，用于管理对小块内存的申请和释放。

Python 内部默认的小块内存与大块内存的分界点定在 256 个字节，当申请的内存小于 256 字节时，PyObject_Malloc 会在内存池中申请内存；当申请的内存大于 256 字节时，PyObject_Malloc 的行为将蜕化为 malloc 的行为。当然，通过修改 Python 源代码，我们可以改变这个默认值，从而改变 Python 的默认内存管理行为。

调优手段（了解）

1.手动垃圾回收

http://www.itheima.com Copyright©2018 黑马程序员

2.调高垃圾回收阈值

3.避免循环引用（手动解循环引用和使用弱引用）

2.内存泄露是什么？如何避免？(2018-3-30-lxy)

指由于疏忽或错误造成程序未能释放已经不再使用的内存的情况。内存泄漏并非指内存在物理上的消失，而是应用程序分配某段内存后，由于设计错误，失去了对该段内存的控制，因而造成了内存的浪费。导致程序运行速度减慢甚至系统崩溃等严重后果。

有 __del__() 函数的对象间的循环引用是导致内存泄漏的主凶。

不使用一个对象时使用:del object 来删除一个对象的引用计数就可以有效防止内存泄漏问题。

通过 Python 扩展模块 gc 来查看不能回收的对象的详细信息。

可以通过 sys.getrefcount(obj) 来获取对象的引用计数，并根据返回值是否为 0 来判断是否内存泄漏。

三．函数

1. 函数参数

1.1 Python 函数调用的时候参数的传递方式是值传递还是引用传递？(2018-3-30-lxy)

Python 的参数传递有：位置参数、默认参数、可变参数、关键字参数。

函数的传值到底是值传递还是引用传递，要分情况：

不可变参数用值传递：

http://www.itheima.com Copyright©2018 黑马程序员

像整数和字符串这样的不可变对象，是通过拷贝进行传递的，因为你无论如何都不可能在原处改变不可变对象

可变参数是引用传递的：

比如像列表，字典这样的对象是通过引用传递、和 C 语言里面的用指针传递数组很相似，可变对象能在函数内部改变。

1.2 对缺省参数的理解 ？(2018-3-30-lxy)

缺省参数指在调用函数的时候没有传入参数的情况下，调用默认的参数，在调用函数的同时赋值时，所传入的参数会替代默认参数。

*args 是不定长参数，他可以表示输入参数是不确定的，可以是任意多个。

**kwargs 是关键字参数，赋值的时候是以键 = 值的方式，参数是可以任意多对在定义函数的时候不确定会有多少参数会传入时，就可以使用两个参数。

1.3 为什么函数名字可以当做参数用?(2018-3-30-lxy)

Python 中一切皆对象，函数名是函数在内存中的空间，也是一个对象。

1.4 Python 中 pass 语句的作用是什么？(2018-3-30-lxy)

在编写代码时只写框架思路，具体实现还未编写就可以用 pass 进行占位，使程序不报错，不会进行任何操作。

1.5 有这样一段代码，print c 会输出什么，为什么？(2018-3-30-lxy)

```
1. a = 10
2. b = 20
3. c = [a]
```


http://www.itheima.com Copyright©2018 黑马程序员

```
4. a = 15
```

答：10 对于字符串、数字，传递是相应的值。

1.6 交换两个变量的值？(2018-4-16-lxy)

```
1. a,b = b,a
```

2. 内建函数

2.1 map 函数和 reduce 函数？ (2018-3-30-lxy)

①从参数方面来讲：

map()包含两个参数，第一个参数是一个函数，第二个是序列（列表 或元组）。其中，函数（即 map 的第一个参数位置的函数）可以接收一个或多个参数。

reduce()第一个参数是函数，第二个是序列（列表或元组）。但是，其函数必须接收两个参数。

②从对传进去的数值作用来讲：

map()是将传入的函数依次作用到序列的每个元素，每个元素都是独自被函数“作用”一次 。

reduce()是将传人的函数作用在序列的第一个元素得到结果后，把这个结果继续与下一个元素作用（累积计算）。

2.2 递归函数停止的条件？(2018-3-30-lxy)

递归的终止条件一般定义在递归函数内部，在递归调用前要做一个条件判断，根据判断的结果选择是继续调用自身，还是 return;返回终止递归。

终止的条件：

1. 判断递归的次数是否达到某一限定值
2. 判断运算的结果是否达到某个范围等，根据设计的目的来选择

Copyright©2018 黑马程序员

2.3 回调函数，如何通信的? (2018-3-30-lxy)

回调函数是把函数的指针(地址)作为参数传递给另一个函数，将整个函数当作一个对象，赋值给调用的函数。

2.4 Python 主要的内置数据类型都有哪些？ print dir(‘a ’) 的输出？(2018-3-30-lxy)

内建类型：布尔类型、数字、字符串、列表、元组、字典、集合；

输出字符串 ‘a’ 的内建方法；

2.5 print(list(map(lambda x: x * x, [y for y in range(3)])))的输出？(2018-3-30-lxy)

```
1. [0 ， 1 ， 4]
```

2.6 hasattr() getattr() setattr() 函数使用详解？(2018-4-16-lxy)

hasattr(object, name)函数：

判断一个对象里面是否有 name 属性或者 name 方法 ,返回 bool 值 ,有 name 属性(方法)返回 True，否则返回 False。

注意：name 要使用引号括起来。

```
1. class function_demo(object):
2.     name = 'demo'
3.     def run(self):
4.         return "hello function"
5. functiondemo = function_demo()
6. res = hasattr(functiondemo, 'name')  #判断对象是否有 name 属性，True
7. res = hasattr(functiondemo, "run") #判断对象是否有 run 方法，True
8. res = hasattr(functiondemo, "age") #判断对象是否有 age 属性，Falsw
9. print(res)
```


Copyright©2018 黑马程序员

getattr(object, name[,default]) 函数：

获取对象 object 的属性或者方法，如果存在则打印出来，如果不存在，打印默认值，默认值可选。

注意：如果返回的是对象的方法，则打印结果是：方法的内存地址，如果需要运行这个方法，可以在后面添加括号()。

```
1. functiondemo = function_demo()
2. getattr(functiondemo, 'name') #获取 name 属性，存在就打印出来--- demo
3. getattr(functiondemo, "run") #获取 run 方法，存在打印出 方法的内存地址---<bound method function_demo.run of <__main__.function_demo object at 0x10244f320>>
4. getattr(functiondemo, "age") #获取不存在的属性，报错如下：
5. Traceback (most recent call last):
6.    File "/Users/liuhuiling/Desktop/MT_code/OpAPIDemo/conf/OPCommUtil.py", line 39, in <module>
7.       res = getattr(functiondemo, "age")
8. AttributeError: 'function_demo' object has no attribute 'age'
9. getattr(functiondemo, "age", 18)   #获取不存在的属性，返回一个默认值
```

setattr(object,name,values)函数：

给对象的属性赋值，若属性不存在，先创建再赋值

```
1．class function_demo(object):
2．       name = 'demo'
3．       def run(self):
4．              return "hello function"
5．functiondemo = function_demo()
6．res = hasattr(functiondemo, 'age')   # 判断 age 属性是否存在，False
7．print(res)
8．setattr(functiondemo, 'age', 18 )   #对 age 属性进行赋值，无返回值
9．res1 = hasattr(functiondemo, 'age') #再次判断属性是否存在，True
```

综合使用：

```
1．class function_demo(object):
2．       name = 'demo'
3．       def run(self):
4．              return "hello function"
5．functiondemo = function_demo()
6．res = hasattr(functiondemo, 'addr') #  先判断是否存在 if res:
7．       addr = getattr(functiondemo, 'addr')
8．       print(addr)else:
9．       addr = getattr(functiondemo, 'addr', setattr(functiondemo, 'addr', '北京首都'))
10．       #addr = getattr(functiondemo, 'addr', '美国纽约')
11．       print(addr)
```


Copyright©2018 黑马程序员

2.7 一句话解决阶乘函数？(2018-4-16-lxy)

在 Python2 中：

```
1．reduce(lambda x,y: x*y, range(1,n+1))
```

注意：Python3 中取消了该函数。

3.Lambda

3.1 什么是 lambda 函数？ 有什么好处？(2018-4-16-lxy)

lambda 函数是一个可以接收任意多个参数(包括可选参数)并且返回单个表达式值的函数

1、lambda 函数比较轻便，即用即仍，很适合需要完成一项功能，但是此功能只在此一处使用，连名字都很随意的情况下；

2、匿名函数，一般用来给 filter， map 这样的函数式编程服务;

3、作为回调函数，传递给某些应用，比如消息处理

3.2 下面这段代码的输出结果将是什么？请解释。(2018-3-30-lxy)

```
1. def multipliers():
2.         return [lambda x : i * x for i in range(4)]
3.         print [m(2) for m in multipliers()]
```

上面代码输出的结果是[6， 6， 6， 6] (不是我们想的[0， 2， 4， 6])。

你如何修改上面的 multipliers 的定义产生想要的结果？

上述问题产生的原因是 Python 闭包的延迟绑定。这意味着内部函数被调用时，参数的值在闭包内进行查找。因此，当任何由 multipliers()返回的函数被调用时，i 的值将在附近的范围进行查找。那时，不管返回的函数是否被调用，for 循环已经完成，i 被赋予了最终的值 3。

Copyright©2018 黑马程序员

因此，每次返回的函数乘以传递过来的值 3，因为上段代码传过来的值是 2，它们最终返回的都是 6。(3*2)碰巧的是，《The Hitchhiker’ s Guide to Python》也指出，在与 lambdas 函数相关也有一个被广泛被误解的知识点，不过跟这个 case 不一样。由 lambda 表达式创造的函数没有什么特殊的地方，它其实是和 def 创造的函数式一样的。

下面是解决这一问题的一些方法。

一种解决方法就是用 Python 生成器。

```
1. def multipliers():
2.         for i in range(4): yield lambda x : i * x
```

另外一个解决方案就是创造一个闭包，利用默认函数立即绑定。

```
1. def   multipliers():
2.    return [lambda x ,  i=i : i * x for i in range(4)]
```

3.3 什么是 lambda 函数？它有什么好处？写一个匿名函数求两个数的和？(2018-3-30-lxy)

lambda 函数是匿名函数；使用 lambda 函数能创建小型匿名函数。这种函数得名于省略了用 def 声明函数的标准步骤；

```
1.    f = lambda x，y:x+y
2.    print(f(2017，2018))
```

四．设计模式

1. 单例

1.1 请手写一个单例(2018-3-30-lxy)

```
1. class A(object):
2.     __instance = None
3.     def __new__(cls ,  *args ,  **kwargs):
4.         if   cls.__instance is None:
5.             cls.__instance = object.__new__(cls)
```


http://www.itheima.com Copyright©2018 黑马程序员

```
6.                 return cls.__instance
7.             else:
8.                 return cls.__instance
```

1.2 单例模式的应用场景有哪些？(2018-4-16-lxy)

单例模式应用的场景一般发现在以下条件下：

（1）资源共享的情况下，避免由于资源操作时导致的性能或损耗等。如日志文件，应用配置。

（2）控制资源的情况下，方便资源之间的互相通信。如线程池等。 1.网站的计数器 2.应用配置 3.多线程池 4.数据库配置，数据库连接池 5.应用程序的日志应用....

2. 工厂

3. 装饰器

3.1 对装饰器的理解 ，并写出一个计时器记录方法执行性能的装饰器？(2018-3-30-lxy)

装饰器本质上是一个 Python 函数 ,它可以让其他函数在不需要做任何代码变动的前提下增加额外功能，装饰器的返回值也是一个函数对象。

```
1.  import time
2.  def timeit(func):
3.      def wrapper():
4.              start = time.clock()
5.              func() end =time.clock()
6.              print 'used:' ,  end - start
7.              return wrapper
8.  @timeit
9.  def foo():
10.         print 'in foo()'foo()
```


Copyright©2018 黑马程序员

3.2 解释一下什么是闭包?(2018-3-30-lxy)

在函数内部再定义一个函数，并且这个函数用到了外边函数的变量，那么将这个函数以及用到的一些变量称之为闭包。

3.3 函数装饰器有什么作用？ (2018-4-16-lxy)

装饰器本质上是一个 Python 函数 ,它可以在让其他函数在不需要做任何代码的变动的前提下增加额外的功能。装饰器的返回值也是一个函数的对象，它经常用于有切面需求的场景。 比如：插入日志、性能测试、事务处理、缓存、权限的校验等场景 有了装饰器就可以抽离出大量的与函数功能本身无关的雷同代码并发并继续使用。

4. 生成器

4.1 生成器、迭代器的区别？(2018-3-30-lxy)

迭代器是一个更抽象的概念，任何对象，如果它的类有 next 方法和 iter 方法返回自己本身，对于 string、list、dict、tuple 等这类容器对象，使用 for 循环遍历是很方便的。在后台 for 语句对容器对象调用 iter()函数，iter()是 python 的内置函数。iter()会返回一个定义了 next()方法的迭代器对象 ,它在容器中逐个访问容器内元素 ,next()也是 python 的内置函数。在没有后续元素时，next()会抛出一个 StopIteration 异常。

生成器（Generator）是创建迭代器的简单而强大的工具。它们写起来就像是正规的函数，只是在需要返回数据的时候使用 yield 语句。每次 next()被调用时，生成器会返回它脱离的位置（它记忆语句最后一次执行的位置和所有的数据值）

区别：生成器能做到迭代器能做的所有事,而且因为自动创建了 iter()和 next()方法,生成器显得特别简洁,而且生成器也是高效的，使用生成器表达式取代列表解析可以同时节省内存。除了创建和保存程序状态的自动方法,当发生器终结时,还会自动抛出 StopIteration 异常。

http://www.itheima.com Copyright©2018 黑马程序员

4.2 X 是什么类型？(2018-3-30-lxy)

```
1.  X = (for  i  in  ramg(10))
```

答：X 是 generator 类型。

4.3 请尝试用“一行代码”实现将 1-N 的整数列表以 3 为单位分组，比如 1-100 分组后为? (2018-4-20-lxy)

```
1.  print([[x for x in range(1，100)][i:i+3] for i in range(0，len(list_a)，3)])
```

4.4Python 中 yield 的用法？(2018-4-16-lxy)

yield 就是保存当前程序执行状态。你用 for 循环的时候，每次取一个元素的时候就会计算一次。用 yield 的函数叫 generator，和 iterator 一样，它的好处是不用一次计算所有元素，而是用一次算一次，可以节省很多空间。generator 每次计算需要上一次计算结果，所以用 yield，否则一 return，上次计算结果就没了。

```
1. >>> def createGenerator():
2. ...     mylist = range(3)
3. ...     for i in mylist:
4. ...         yield i*i
5. ...
6. >>> mygenerator = createGenerator() # create a generator
7. >>> print(mygenerator) # mygenerator is an object!
8. <generator object createGenerator at 0xb7555c34>
9. >>> for i in mygenerator:
10. ...     print(i)
11. 0
12. 1
13. 4
```


Copyright©2018 黑马程序员

五．面向对象

1. 类

2. 对象

2.1 Python 中的可变对象和不可变对象？(2018-3-30-lxy)

不可变对象，该对象所指向的内存中的值不能被改变。当改变某个变量时候，由于其所指的值不能被改变，相当于把原来的值复制一份后再改变，这会开辟一个新的地址，变量再指向这个新的地址。

可变对象，该对象所指向的内存中的值可以被改变。变量（准确的说是引用）改变后，实际上是其所指的值直接发生改变，并没有发生复制行为，也没有开辟新的出地址，通俗点说就是原地改变。

Python 中，数值类型（int 和 float）、字符串 str、元组 tuple 都是不可变类型。而列表 list、字典 dict、集合 set 是可变类型。

2.2 Python 中 is 和==的区别？(2018-3-30-lxy)

is 判断的是 a 对象是否就是 b 对象，是通过 id 来判断的。

==判断的是 a 对象的值是否和 b 对象的值相等，是通过 value 来判断的。

2.3 Python 的魔法方法 (2018-3-30-lxy)

魔法方法就是可以给你的类增加魔力的特殊方法，如果你的对象实现 （重载）了这些方法中的某一个，那么这个方法就会在特殊的情况下被 Python 所调用，你可以定义自己想要的行为，而这一切都是自动发生的。 它们经常是两个下划线包围来命名的（比如 __init__，__lt__），Python 的魔法方法是非常强大的，所以了解其使用方法也变得尤为重要！

Copyright©2018 黑马程序员

__init__ 构造器，当一个实例被创建的时候初始化的方法。但是它并 不是实例化调用的第一个方法。

__new__才是实例化对象调用的第一个方法，它只取下 cls 参数，并把 其他参数传给 __init__。 __new__很少使用，但是也有它适合的场景，尤其 是当类继承自一个像元组或者字符串这样不经常改变的类型的时候。

__call__ 允许一个类的实例像函数一样被调用 。

__getitem__ 定义获取容器中指定元素的行为，相当于 self[key] 。

__getattr__ 定义当用户试图访问一个不存在属性的时候的行为 。

__setattr__ 定义当一个属性被设置的时候的行为 。

__getattribute__ 定义当一个属性被访问的时候的行为 。

2.4 面向对象中怎么实现只读属性? (2018-3-30-lxy)

将对象私有化，通过共有方法提供一个读取数据的接口。

```
1．class person:
2．    def __init__(self,x):
3．        self.__age = 10;
4．    def age(self):
5．        return self.__age;
6．t = person(22)
7．# t.__age = 100
8．print(t.age())
```

最好的方法

```
9．class MyCls(object):
10．    __weight = 50
11．
12．    @property  #以访问属性的方式来访问 weight 方法
13．    def weight(self):
14．        return self.__weight
15．
16．if __name__ == '__main__':
17．    obj = MyCls()
18．    print(obj.weight)
19．    obj.weight = 12
```


Copyright©2018 黑马程序员

```
1．Traceback (most recent call last):
2．50
3．  File "C:/PythonTest/test.py", line 11, in <module>
4．    obj.weight = 12
5．AttributeError: can't set attribute
```

2.5 谈谈你对面向对象的理解？

面向对象是相对于面向过程而言的。面向过程语言是一种基于功能分析的、以算法为中心的程序设计方法；而面向对象是一种基于结构分析的、以数据为中心的程序设计思想。在面向对象语言中有一个有很重要东西，叫做类。

面向对象有三大特性：封装、继承、多态。

六．正则表达式

1. Python 里 match 与 search 的区别？(2018-3-30-lxy)

match()函数只检测 RE 是不是在 string 的开始位置匹配，

search()会扫描整个 string 查找匹配；

也就是说 match()只有在 0 位置匹配成功的话才有返回，

如果不是开始位置匹配成功的话，match()就返回 none。

2. Python 字符串查找和替换？(2018-3-30-lxy)

```
1． re.findall(r’目的字符串’，’原有字符串’) #查询
2． re.findall(r'cast'，'itcast.cn')[0]
3． re.sub(r‘要替换原字符’，’要替换新字符’，’原始字符串’)
4． re.sub(r'cast'，'heima'，'itcast.cn')
```


Copyright©2018 黑马程序员

3. 用 Python 匹配 HTML g tag 的时候，<.*> 和 <.*?> 有什么区别？(2018-3-30-lxy)

<.*>是贪婪匹配，会从第一个“<”开始匹配，直到最后一个“>”中间所有的字符都会匹配到，中间可能会包含“<>”。

<.*?>是非贪婪匹配，从第一个“<”开始往后，遇到第一个“>”结束匹配，这中间的字符串都会匹配到，但是不会有“<>”。

4.请写出下列正则关键字的含义？(2018-3-30-lxy)

语法	说明	表达式实例	完整匹配的字符串
字符			
一般字符	匹配自身	abc	abc
.	匹配任意除换行符"\n"外的字符。在 DOTALL 模式中也能匹配换行符。	a.c	abc
\	转义字符,使后一个字符改变原来的意思。 如果字符串中有字符*需要匹配,可以使用*或者字符集[*]	a\.c a\\c	a. c a\c
[...]	字符集(字符类)。对应的位置可以是字符集中任意字符。字符集中的字符可以逐个列出,也可以给出范围,如[abc]或[a-c]。第一个字符如果是^则表示取反,如[^abc]表示不是 abc 的其他字符。 所有的特殊字符在字符集中都失去其原有的特殊含义。在字符集中如果要使用]、-或^,可以在前面加上反斜杠,或把]、-放在第一个字符,把^放在非第一个字符。	a[bcd]e	abe ace ade
预定义字符集(可以写在字符集[…]中)			
\d	数字:[0-9]	a\dc	a1c
\D	非数字:[^\d]	a\Dc	abc

http://www.itheima.com Copyright©2018 黑马程序员

\s	空白字符:[<空格>\ t\r\n\f\v]	a\sc	ac
\S	非空白字符:[^\s]	a\Sc	abc
\w	单词字符:[A-Za-z0-9_]	a\wc	abc
\W	非单词字符:[^\W]	a\Wc	ac
数量词(用在字符或(…)之后)			
*	匹配前一个字符 0 或无限次。	abc*	ab abccc
+	匹配前一个字符 1 次或无限次	abc+	abc abccc
?	匹配前一个字符 0 次或 1 次。	abc?	Ab abc
{m}	匹配前一个字符 m 次	ab{2}c	abbc

七．系统编程

1. 进程总结

进程：程序运行在操作系统上的一个实例，就称之为进程。进程需要相应的系统资源：内存、时间片、pid。

创建进程：

1.首先要导入 multiprocessing 中的 Process；

2.创建一个 Process 对象；

3.创建 Process 对象时，可以传递参数；

```
1．p = Process(target=XXX, args=(元组,) , kwargs={key:value})
2．target = XXX 指定的任务函数,不用加()
3．args=(元组,) , kwargs={key:value} 给任务函数传递的参数
```

4.使用 start()启动进程；

5.结束进程。

Process 语法结构：

http://www.itheima.com Copyright©2018 黑马程序员

Process([group [, target [, name [, args [, kwargs]]]]])

target：如果传递了函数的引用，可以让这个子进程就执行函数中的代码

args：给 target 指定的函数传递的参数，以元组的形式进行传递

kwargs：给 target 指定的函数传递参数，以字典的形式进行传递

name：给进程设定一个名字，可以省略

group：指定进程组，大多数情况下用不到

Process 创建的实例对象的常用方法有：

start()：启动子进程实例(创建子进程)

is_alive()：判断进程子进程是否还在活着

join(timeout)：是否等待子进程执行结束，或者等待多少秒

terminate()：不管任务是否完成，立即终止子进程

Process 创建的实例对象的常用属性：

name：当前进程的别名，默认为 Process-N,N 为从 1 开始递增的整数

pid：当前进程的 pid(进程号)

给子进程指定函数传递参数 Demo：

```
1．import osfrom multiprocessing import Process
2．import time
3．
4．def pro_func(name, age, **kwargs):
5．    for i in range(5):
6．        print("子进程正在运行中,name=%s, age=%d, pid=%d" %(name, age, os.getpid()))
7．        print(kwargs)
8．        time.sleep(0.2)
9．
10．if __name__ == '__main__':
11．    # 创建 Process 对象
12．    p = Process(target=pro_func, args=('小明',18), kwargs={'m': 20})
13．    # 启动进程
14．    p.start()
```


http://www.itheima.com Copyright©2018 黑马程序员

```
15．    time.sleep(1)
16．    # 1 秒钟之后，立刻结束子进程
17．    p.terminate()
18．    p.join()
```

注意：进程间不共享全局变量。

进程之间的通信-Queue

在初始化 Queue()对象时，(例如 q=Queue()，若在括号中没有指定最大可接受的消息数量，或数量为负值时，那么就代表可接受的消息数量没有上限-直到内存的尽头)

Queue.qsize()：返回当前队列包含的消息数量。

Queue.empty()：如果队列为空，返回 True,反之 False。

Queue.full()：如果队列满了，返回 True，反之 False。

Queue.get([block[,timeout]])：获取队列中的一条消息，然后将其从队列中移除，block 默认值为 True。

如果 block 使用默认值，且没有设置 timeout（单位秒），消息列队如果为空，此时程序将被阻塞（停在读取状态），直到从消息列队读到消息为止，如果设置了 timeout，则会等待 timeout 秒，若还没读取到任何消息，则抛出"Queue.Empty"异常；

如果 block 值为 False，消息列队如果为空，则会立刻抛出"Queue.Empty"异常；

Queue.get_nowait()：相当 Queue.get(False)；

Queue.put(item,[block[, timeout]])：将 item 消息写入队列，block 默认值为 True；

如果 block 使用默认值，且没有设置 timeout（单位秒），消息列队如果已经没有空间可写入，此时程序将被阻塞（停在写入状态），直到从消息列队腾出空间为止，如果设置了 timeout，则会等待 timeout 秒，若还没空间，则抛出"Queue.Full"异常；

如果 block 值为 False，消息列队如果没有空间可写入，则会立刻抛出"Queue.Full"异常；

Queue.put_nowait(item)：相当 Queue.put(item, False)；

http://www.itheima.com Copyright©2018 黑马程序员

进程间通信 Demo：

```
1．from multiprocessing import Process, Queueimport os, time, random
2．# 写数据进程执行的代码:def write(q):
3．    for value in ['A', 'B', 'C']:
4．        print('Put %s to queue...' % value)
5．        q.put(value)
6．        time.sleep(random.random())
7．# 读数据进程执行的代码:def read(q):
8．    while True:
9．        if not q.empty():
10．            value = q.get(True)
11．            print('Get %s from queue.' % value)
12．            time.sleep(random.random())
13．        else:
14．            break
15．if __name__=='__main__':
16．    # 父进程创建 Queue，并传给各个子进程：
17．    q = Queue()
18．    pw = Process(target=write, args=(q,))
19．    pr = Process(target=read, args=(q,))
20．    # 启动子进程 pw，写入:
21．    pw.start()
22．    # 等待 pw 结束:
23．    pw.join()
24．    # 启动子进程 pr，读取:
25．    pr.start()
26．    pr.join()
27．    # pr 进程里是死循环，无法等待其结束，只能强行终止:
28．    print('')
29．    print('所有数据都写入并且读完')
```

进程池 Pool

```
1．# -*- coding:utf-8 -*-
2．from multiprocessing import Poolimport os, time, random
3．def worker(msg):
4．    t_start = time.time()
5．    print("%s 开始执行,进程号为%d" % (msg,os.getpid()))
6．    # random.random()随机生成 0~1 之间的浮点数
7．    time.sleep(random.random()*2)
8．    t_stop = time.time()
9．    print(msg,"执行完毕，耗时%0.2f" % (t_stop-t_start))
10．
11．po = Pool(3)  # 定义一个进程池，最大进程数 3
```


Copyright©2018 黑马程序员

```
12．for i in range(0,10):
13．      # Pool().apply_async(要调用的目标,(传递给目标的参数元祖,))
14．      # 每次循环将会用空闲出来的子进程去调用目标
15．      po.apply_async(worker,(i,))
16．
17．print("----start----")
18．po.close()  # 关闭进程池，关闭后 po 不再接收新的请求
19．po.join()  # 等待 po 中所有子进程执行完成，必须放在 close 语句之后
20．print("-----end-----")
```

multiprocessing.Pool 常用函数解析：

- apply_async(func[, args[, kwds]]) ：使用非阻塞方式调用 func（并行执行，堵塞方式必须等待上一个进程退出才能执行下一个进程），args 为传递给 func 的参数列表，kwds 为传递给 func 的关键字参数列表；
- close()：关闭 Pool，使其不再接受新的任务；
- terminate()：不管任务是否完成，立即终止；
- join()：主进程阻塞，等待子进程的退出， 必须在 close 或 terminate 之后使用；

进程池中使用 Queue

如果要使用 Pool 创建进程，就需要使用 multiprocessing.Manager()中的 Queue()，而不是 multiprocessing.Queue()，否则会得到一条如下的错误信息：

RuntimeError: Queue objects should only be shared between processes through inheritance.

```
1．from multiprocessing import Manager,Poolimport os,time,random
2．def reader(q):
3．      print("reader 启动(%s),父进程为(%s)" % (os.getpid(), os.getppid()))
4．      for i in range(q.qsize()):
5．            print("reader 从 Queue 获取到消息：%s" % q.get(True))
6．def writer(q):
7．      print("writer 启动(%s),父进程为(%s)" % (os.getpid(), os.getppid()))
8．      for i in "itcast":
9．            q.put(i)
10．if __name__=="__main__":
11．      print("(%s) start" % os.getpid())
```


Copyright©2018 黑马程序员

```
    q = Manager().Queue()  # 使用 Manager 中的 Queue
    po = Pool()
    po.apply_async(writer, (q,))

    time.sleep(1)  # 先让上面的任务向 Queue 存入数据，然后再让下面的任务开始从中取数据

    po.apply_async(reader, (q,))
    po.close()
    po.join()
    print("(%s) End" % os.getpid())
```

2. 谈谈你对多进程，多线程，以及协程的理解，项目是否用？(2018-3-30-lxy)

这个问题被问的概率相当之大，其实多线程，多进程，在实际开发中用到的很少，除非是那些对项目性能要求特别高的，有的开发工作几年了，也确实没用过，你可以这么回答，给他扯扯什么是进程，线程（cpython 中是伪多线程）的概念就行，实在不行你就说你之前写过下载文件时，用过多线程技术，或者业余时间用过多线程写爬虫，提升效率。

进程：一个运行的程序（代码）就是一个进程，没有运行的代码叫程序，进程是系统资源分配的最小单位，进程拥有自己独立的内存空间，所以进程间数据不共享，开销大。

线程： 调度执行的最小单位，也叫执行路径，不能独立存在，依赖进程存在一个进程至少有一个线程，叫主线程，而多个线程共享内存(数据共享，共享全局变量)，从而极大地提高了程序的运行效率。

协程：是一种用户态的轻量级线程，协程的调度完全由用户控制。协程拥有自己的寄存器上下文和栈。 协程调度切换时，将寄存器上下文和栈保存到其他地方，在切回来的时候，恢复先前保存的寄存器上下文和栈，直接操作栈则基本没有内核切换的开销，可以不加锁的访问全局变量，所以上下文的切换非常快。

Copyright©2018 黑马程序员

3. 什么是多线程竞争？(2018-3-30-lxy)

线程是非独立的，同一个进程里线程是数据共享的，当各个线程访问数据资源时会出现竞争状态即：数据几乎同步会被多个线程占用，造成数据混乱 ，即所谓的线程不安全

那么怎么解决多线程竞争问题？-- 锁。

锁的好处：

确保了某段关键代码(共享数据资源)只能由一个线程从头到尾完整地执行能解决多线程资源竞争下的原子操作问题。

锁的坏处：

阻止了多线程并发执行，包含锁的某段代码实际上只能以单线程模式执行，效率就大大地下降了

锁的致命问题：死锁。

4. 解释一下什么是锁，有哪几种锁? (2018-3-30-lxy)

锁(Lock)是 Python 提供的对线程控制的对象。有互斥锁、可重入锁、死锁。

5. 什么是死锁呢？(2018-3-30-lxy)

若干子线程在系统资源竞争时，都在等待对方对某部分资源解除占用状态，结果是谁也不愿先解锁，互相干等着，程序无法执行下去，这就是死锁。

GIL 锁（有时候，面试官不问，你自己要主动说，增加 b 格，尽量别一问一答的尬聊，不然最后等到的一句话就是：你还有什么想问的么？ ）

GIL 锁 全局解释器锁（只在 cpython 里才有）

作用：限制多线程同时执行，保证同一时间只有一个线程执行，所以 cpython 里的多线程其实是伪多线程!

http://www.itheima.com Copyright©2018 黑马程序员

所以 Python 里常常使用协程技术来代替多线程，协程是一种更轻量级的线程，

进程和线程的切换时由系统决定，而协程由我们程序员自己决定，而模块 gevent 下切换是遇到了耗时操作才会切换。

三者的关系：进程里有线程，线程里有协程。

6. 什么是线程安全，什么是互斥锁？(2018-3-30-lxy)

每个对象都对应于一个可称为" 互斥锁" 的标记，这个标记用来保证在任一时刻，只能有一个线程访问该对象。

同一个进程中的多线程之间是共享系统资源的，多个线程同时对一个对象进行操作，一个线程操作尚未结束，另一个线程已经对其进行操作，导致最终结果出现错误，此时需要对被操作对象添加互斥锁，保证每个线程对该对象的操作都得到正确的结果。

7. 说说下面几个概念：同步，异步，阻塞，非阻塞?(2018-3-30-lxy)

同步：多个任务之间有先后顺序执行，一个执行完下个才能执行。

异步：多个任务之间没有先后顺序，可以同时执行有时候一个任务可能要在必要的时候获取另一个同时执行的任务的结果，这个就叫回调！

阻塞：如果卡住了调用者，调用者不能继续往下执行，就是说调用者阻塞了。

非阻塞：如果不会卡住，可以继续执行，就是说非阻塞的。

同步异步相对于多任务而言，阻塞非阻塞相对于代码执行而言。

Copyright©2018 黑马程序员

8. 什么是僵尸进程和孤儿进程？怎么避免僵尸进程? (2018-3-30-lxy)

孤儿进程：父进程退出，子进程还在运行的这些子进程都是孤儿进程，孤儿进程将被 init 进程(进程号为 1)所收养，并由 init 进程对它们完成状态收集工作。

僵尸进程 :进程使用 fork 创建子进程 ,如果子进程退出 ,而父进程并没有调用 wait 或 waitpid 获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中的这些进程是僵尸进程。

避免僵尸进程的方法：

1.fork 两次用孙子进程去完成子进程的任务；

2.用 wait()函数使父进程阻塞；

3.使用信号量，在 signal handler 中调用 waitpid，这样父进程不用阻塞。

9. Python 中的进程与线程的使用场景? (2018-3-30-lxy)

多进程适合在 CPU 密集型操作(cpu 操作指令比较多，如位数多的浮点运算)。

多线程适合在 IO 密集型操作(读写数据操作较多的，比如爬虫)。

10.线程是并发还是并行，进程是并发还是并行？(2018-3-30-lxy)

线程是并发，进程是并行；

进程之间相互独立，是系统分配资源的最小单位，同一个线程中的所有线程共享资源。

11.并行（parallel）和并发（concurrency）？(2018-3-30-lxy)

并行：同一时刻多个任务同时在运行。

并发：在同一时间间隔内多个任务都在运行，但是并不会在同一时刻同时运行，存在交替执行的情况。

Copyright©2018 黑马程序员

实现并行的库有：multiprocessing

实现并发的库有：threading

程序需要执行较多的读写、请求和回复任务的需要大量的 IO 操作，IO 密集型操作使用并发更好。

CPU 运算量大的程序程序，使用并行会更好。

12.IO 密集型和 CPU 密集型区别？(2018-4-16-lxy)

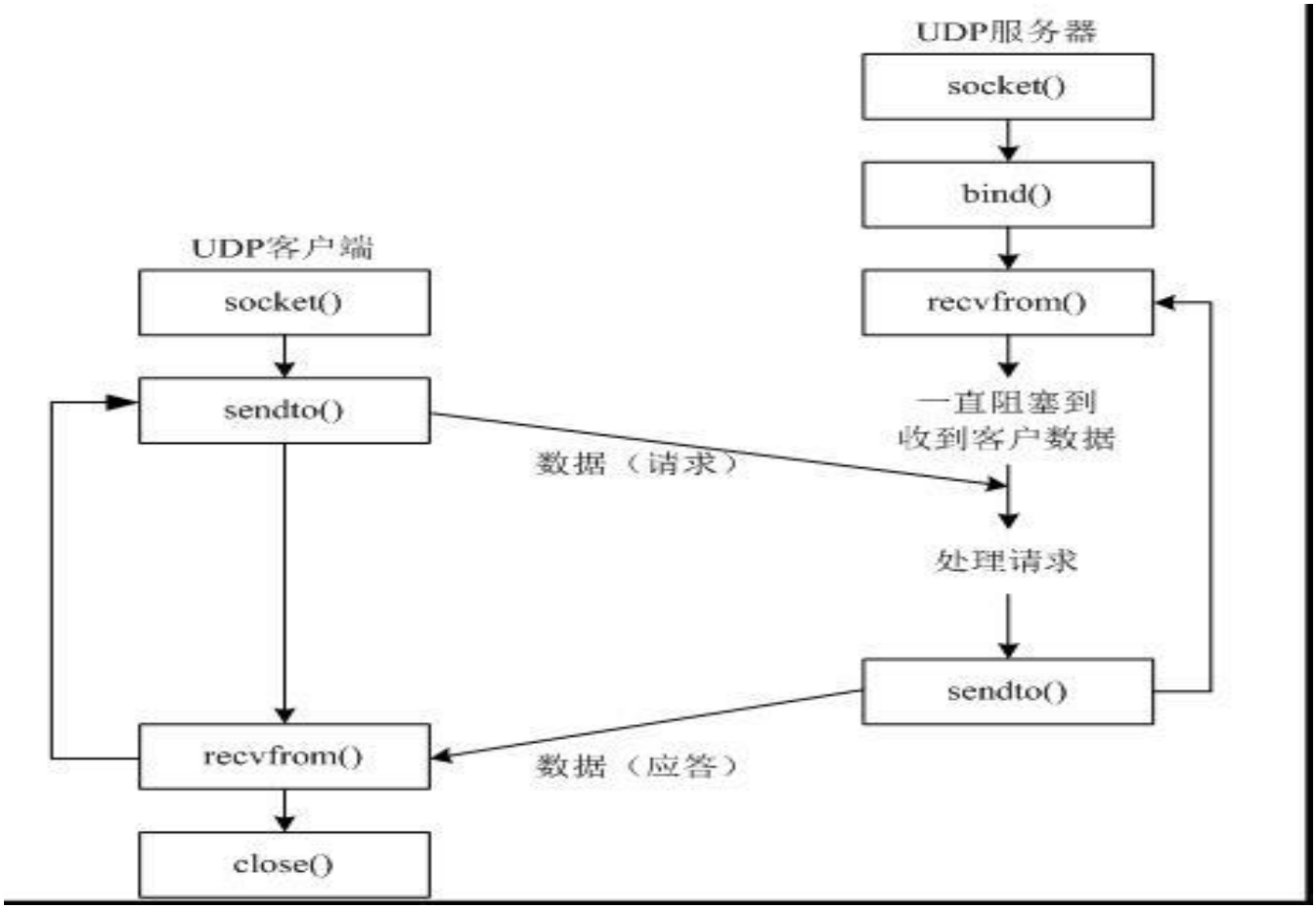
IO 密集型：系统运作，大部分的状况是 CPU 在等 I/O (硬盘/内存)的读/写。

CPU 密集型：大部份时间用来做计算、逻辑判断等 CPU 动作的程序称之 CPU 密集型。

Copyright©2018 黑马程序员

八．网络编程

1. UDP 总结



使用 udp 发送/接收数据步骤：

1.创建客户端套接字

2.发送/接收数据

3.关闭套接字

```
1．import socket
2．def main():
3．    # 1、创建 udp 套接字
4．    # socket.AF_INET  表示 IPv4 协议  AF_INET6 表示 IPv6 协议
5．    # socket.SOCK_DGRAM  数据报套接字，只要用于 udp 协议
6．    udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
7．
```


Copyright©2018 黑马程序员

```
    # 2、准备接收方的地址
    # 元组类型  ip 是字符串类型   端口号是整型
    dest_addr = ('192.168.113.111', 8888)
    # 要发送的数据
    send_data = "我是要发送的数据"
    # 3、发送数据
    udp_socket.sendto(send_data.encode("utf-8"), dest_addr)
    # 4、等待接收方发送的数据  如果没有收到数据则会阻塞等待，直到收到数据
    # 接收到的数据是一个元组   (接收到的数据, 发送方的 ip 和端口)
    # 1024  表示本次接收的最大字节数
    recv_data, addr = udp_socket.recvfrom(1024)
    # 5、关闭套接字
    udp_socket.close()
if __name__ == '__main__':
    main()
```

编码的转换

str -->bytes: encode 编码

bytes--> str: decode()解码

UDP 绑定端口号：

1.创建 socket 套接字

2.绑定端口号

3.接收/发送数据

4.关闭套接字

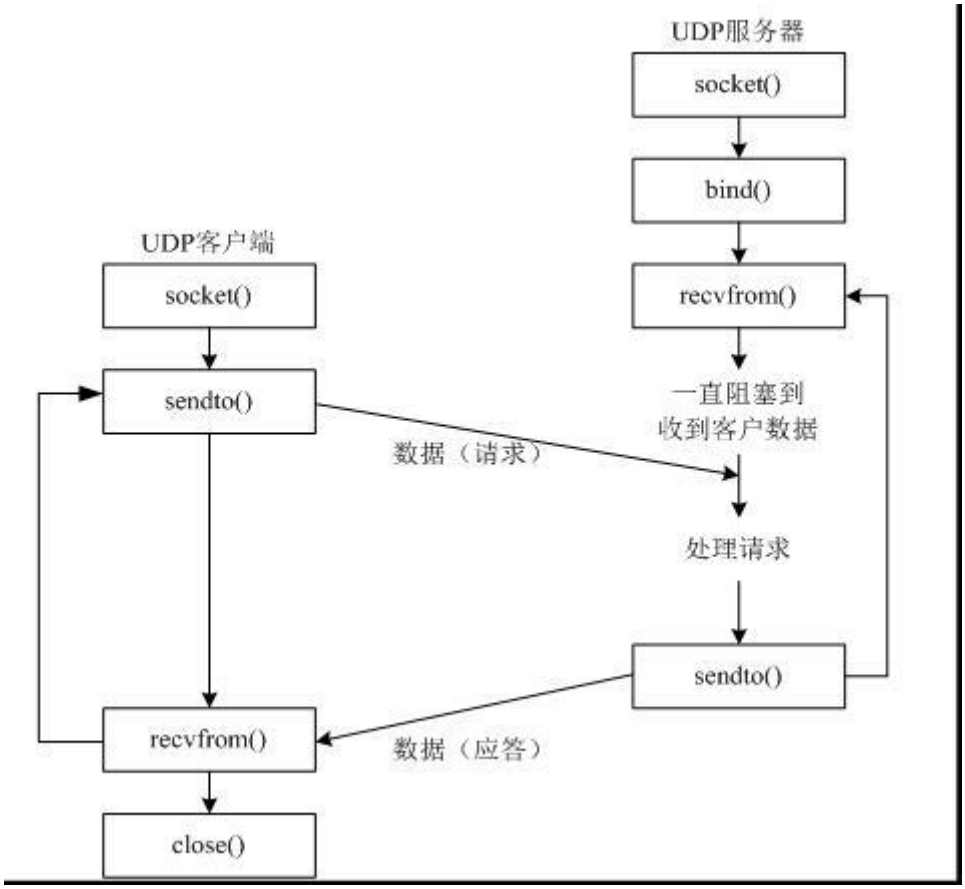
```
import socket
def main():
    # 1、创建 udp 套接字
    # socket.AF_INET  表示 IPv4 协议  AF_INET6 表示 IPv6 协议
    # socket.SOCK_DGRAM  数据报套接字，只要用于 udp 协议
    udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    # 2、绑定端口
    # 元组类型  ip 一般不写 表示本机的任何的一个 ip
    local_addr = ('', 7777)
    udp_socket.bind(local_addr)
    # 3、准备接收方的地址
    # 元组类型  ip 是字符串类型   端口号是整型
    dest_addr = ('192.168.113.111', 8888)
```


Copyright©2018 黑马程序员

```
14．      # 要发送的数据
15．      send_data = "我是要发送的数据"
16．      # 4、发送数据
17．      udp_socket.sendto(send_data.encode("utf-8"), dest_addr)
18．      # 5、等待接收方发送的数据  如果没有收到数据则会阻塞等待，直到收到数据
19．      # 接收到的数据是一个元组    (接收到的数据, 发送方的 ip 和端口)
20．      # 1024  表示本次接收的最大字节数
21．      recv_data, addr = udp_socket.recvfrom(1024)
22．      # 6、关闭套接字
23．      udp_socket.close()
24．if __name__ == '__main__':
25．      main()
```

注意点：绑定端口要在发送数据之前进行绑定。

2. TCP 总结



TCP 客户端的创建流程：

1.创建 TCP 的 socket 套接字

http://www.itheima.com Copyright©2018 黑马程序员

2.连接服务器

3.发送数据给服务器端

4.接收服务器端发送来的消息

5.关闭套接字

```
1．import socket
2．def main():
3．    # 1、创建客户端的 socket
4．    # socket.AF_INET   表示 IPv4 协议   AF_INET6  表示 IPv6 协议
5．    # socket.SOCK_STREAM   流式套接字，只要用于 TCP 协议
6．    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7．    # 2、构建目标地址
8．    server_ip = input("请输入服务器端的 IP 地址：")
9．    server_port = int(input("请输入服务器端的端口号："))
10．    # 3、连接服务器
11．    # 参数：元组类型    ip 是字符串类型    端口号是整型
12．    client_socket.connect((server_ip, server_port))
13．    # 要发送给服务器端的数据
14．    send_data = "我是要发送给服务器端的数据"
15．    # 4、发送数据
16．    client_socket.send(send_data.encode("gbk"))
17．    # 5、接收服务器端恢复的消息， 没有消息会阻塞
18．    # 1024 表示接收的最大字节数
19．    recv_date= client_socket.recv(1024)
20．    print("接收到的数据是：", recv_date.decode('gbk'))
21．    # 6、关闭套接字
22．    client_socket.close()
23．if __name__ == '__main__':
24．    main()
```

TCP 服务器端的创建流程

1.创建 TCP 服务端的 socket

2.bing 绑定 ip 地址和端口号

3.listen 使套接字变为被动套接字

4.accept 取出一个客户端连接，用于服务

5.recv/send 接收和发送消息

Copyright©2018 黑马程序员

6.关闭套接字

```
1．import socket
2．
3．def main():
4．    # 1、创建 tcp 服务端的 socket
5．    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6．
7．    # 2、绑定
8．    server_socket.bind(('', 8888))
9．
10．    # 3、listen 使套接字变为被动套接字
11．    server_socket.listen(128)
12．
13．    # 4、如果有新的客户端来链接服务器，那么就产生一个新的套接字专门为这个客户端服务
14．    # client_socket 用来为这个客户端服务
15．    # tcp_server_socket 就可以省下来专门等待其他新客户端的链接
16．    client_socket, client_addr = server_socket.accept()
17．
18．    # 5、接收客户端发来的消息
19．    recv_data = client_socket.recv(1024)
20．    print("接收到客户端%s 的数据：%s" % (str(client_addr), recv_data.decode('gbk')))
21．
22．    # 6、回复数据给客户端
23．    client_socket.send("收到消息".encode('gbk'))
24．
25．    # 7、关闭套接字
26．    client_socket.close()
27．    server_socket.close()
28．
29．if __name__ == '__main__':
30．    main()
```

注意点：

- tcp 服务器一般都需要绑定，否则客户端找不到服务器
- tcp 客户端一般不绑定，因为是主动链接服务器，所以只要确定好服务器的 ip、port 等信息就好，本地客户端可以随机
- tcp 服务器中通过 listen 可以将 socket 创建出来的主动套接字变为被动的，这是做 tcp 服务器时必须要做的

Copyright©2018 黑马程序员

- 当客户端需要链接服务器时，就需要使用 connect 进行链接，udp 是不需要链接的而是直接发送，但是 tcp 必须先链接，只有链接成功才能通信
- 当一个 tcp 客户端连接服务器时，服务器端会有 1 个新的套接字，这个套接字用来标记这个客户端，单独为这个客户端服务
- listen 后的套接字是被动套接字，用来接收新的客户端的连接请求的，而 accept 返回的新套接字是标识这个新客户端的
- 关闭 listen 后的套接字意味着被动套接字关闭了，会导致新的客户端不能够链接服务器，但是之前已经链接成功的客户端正常通信。
- 关闭 accept 返回的套接字意味着这个客户端已经服务完毕
- 当客户端的套接字调用 close 后，服务器端会 recv 解阻塞，并且返回的长度为 0，因此服务器可以通过返回数据的长度来区别客户端是否已经下线；同理 当服务器断开 tcp 连接的时候 客户端同样也会收到 0 字节数据。

3. 怎么实现强行关闭客户端和服务器之间的连接? (2018-3-30-lxy)

在 socket 通信过程中不断循环检测一个全局变量(开关标记变量)，一旦标记变量变为关闭，则 调用 socket 的 close 方法，循环结束，从而达到关闭连接的目的。

4. 简述 TCP 和 UDP 的区别以及优缺点? (2018-4-16-lxy)

UDP 是面向无连接的通讯协议，UDP 数据包括目的端口号和源端口号信息。

优点：UDP 速度快、操作简单、要求系统资源较少，由于通讯不需要连接，可以实现广播发送

缺点：UDP 传送数据前并不与对方建立连接，对接收到的数据也不发送确认信号，发送端不知道数据是否会正确接收，也不重复发送，不可靠。

Copyright©2018 黑马程序员

TCP 是面向连接的通讯协议，通过三次握手建立连接，通讯完成时四次挥手

优点：TCP 在数据传递时，有确认、窗口、重传、阻塞等控制机制，能保证数据正确性，较为可靠。

缺点：TCP 相对于 UDP 速度慢一点，要求系统资源较多。

5. 简述浏览器通过 WSGI 请求动态资源的过程? (2018-4-16-lxy)

1.发送 http 请求动态资源给 web 服务器

2.web 服务器收到请求后通过 WSGI 调用一个属性给应用程序框架

3.应用程序框架通过引用 WSGI 调用 web 服务器的方法，设置返回的状态和头信息。

4.调用后返回，此时 web 服务器保存了刚刚设置的信息

5.应用程序框架查询数据库，生成动态页面的 body 的信息

6.把生成的 body 信息返回给 web 服务器

7.web 服务器吧数据返回给浏览器

6. 描述用浏览器访问 www.baidu.com 的过程(2018-4-16-lxy)

先要解析出 baidu.com 对应的 ip 地址

- 要先使用 arp 获取默认网关的 mac 地址
- 组织数据发送给默认网关(ip 还是 dns 服务器的 ip，但是 mac 地址是默认网关的 mac 地址)
- 默认网关拥有转发数据的能力，把数据转发给路由器
- 路由器根据自己的路由协议，来选择一个合适的较快的路径转发数据给目的网关
- 目的网关(dns 服务器所在的网关)，把数据转发给 dns 服务器
- dns 服务器查询解析出 baidu.com 对应的 ip 地址，并原路返回请求这个域名的 client

得到了 baidu.com 对应的 ip 地址之后，会发送 tcp 的 3 次握手，进行连接

Copyright©2018 黑马程序员

- 使用 http 协议发送请求数据给 web 服务器
- web 服务器收到数据请求之后，通过查询自己的服务器得到相应的结果，原路返回给浏览器。
- 浏览器接收到数据之后通过浏览器自己的渲染功能来显示这个网页。
- 浏览器关闭 tcp 连接，即 4 次挥手结束，完成整个访问过程

7. Post 和 Get 请求的区别? (2018-4-16-lxy)

GET 请求，请求的数据会附加在 URL 之后，以?分割 URL 和传输数据，多个参数用&连接。URL 的编码格式采用的是 ASCII 编码，而不是 uniclde，即是说所有的非 ASCII 字符都要编码之后再传输。

POST 请求：POST 请求会把请求的数据放置在 HTTP 请求包的包体中。上面的 item=bandsaw 就是实际的传输数据。

因此，GET 请求的数据会暴露在地址栏中，而 POST 请求则不会。

传输数据的大小：

- 在 HTTP 规范中，没有对 URL 的长度和传输的数据大小进行限制。但是在实际开发过程中，对于 GET，特定的浏览器和服务器对 URL 的长度有限制。因此，在使用 GET 请求时，传输数据会受到 URL 长度的限制。
- 对于 POST ,由于不是 URL 传值 ,理论上是不会受限制的 ,但是实际上各个服务器会规定对 POST 提交数据大小进行限制，Apache、IIS 都有各自的配置。

安全性：

- POST 的安全性比 GET 的高。这里的安全是指真正的安全，而不同于上面 GET 提到的安全方法中的安全 ,上面提到的安全仅仅是不修改服务器的数据。比如 ,在进行登录操作 ,通过 GET 请求，用户名和密码都会暴露再 URL 上，因为登录页面有可能被浏览器缓存以及其他人查看浏览器的

Copyright©2018 黑马程序员

历史记录的原因，此时的用户名和密码就很容易被他人拿到了。除此之外，GET 请求提交的数据还可能会造成 Cross-site request frogery 攻击。

效率：GET 比 POST 效率高。

POST 请求的过程：

1.浏览器请求 tcp 连接（第一次握手）

2.服务器答应进行 tcp 连接（第二次握手）

3.浏览器确认，并发送 post 请求头（第三次握手，这个报文比较小，所以 http 会在此时进行第一次数据发送）

4.服务器返回 100 continue 响应

5.浏览器开始发送数据

6.服务器返回 200 ok 响应

GET 请求的过程：

1.浏览器请求 tcp 连接（第一次握手）

2.服务器答应进行 tcp 连接（第二次握手）

3.浏览器确认，并发送 get 请求头和数据（第三次握手，这个报文比较小，所以 http 会在此时进行第一次数据发送）

4.服务器返回 200 OK 响应

8. cookie 和 session 的区别？(2018-4-16-lxy)

1、cookie 数据存放在客户的浏览器上，session 数据放在服务器上。

2、cookie 不是很安全，别人可以分析存放在本地的 cookie 并进行 cookie 欺骗考虑到安全应当使用 session。

http://www.itheima.com Copyright©2018 黑马程序员